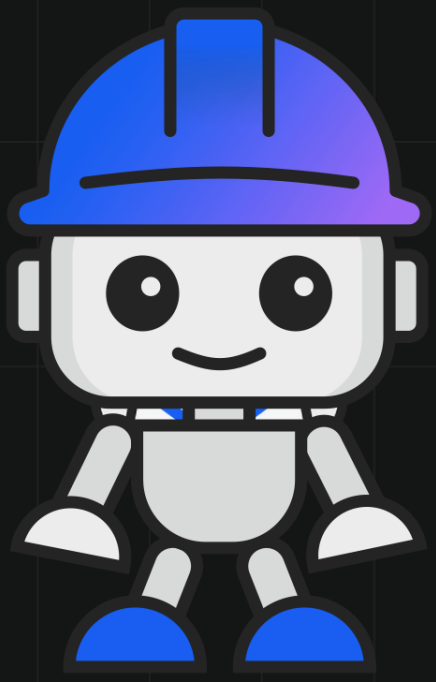
IBM
DEVELOPER SERIES

GETTING STARTED WITH IBM BOB



WHAT YOU'LL LEARN

- → AI-assisted development
- → Skills & agent patterns
- → Enterprise Java workflows

```
// bob.properties
role = "sdlc-partner"
approach = "understand-first"
control = "developer"
mode = "context-aware"
target = "real-codebases"
```

```
import com.ibm.bob.agent.AgentSkill;
import com.ibm.bob.skills.SkillContext;

@AgentSkill(name = "swiftship-deploy")
public class DeploySkill implements Skill {
public SkillResult execute(SkillCtx ctx) {
return SkillResult.success(ctx);
}
```

Experience. Judgment. Systems.

MARKUS EISELE

Getting Started with IBM Bob

Markus Eisele

Version 1.2.0

Table of Contents

Copyright and IBM Bob

IBM Bob is IBM's AI-powered partner for the software development lifecycle—supporting work in the IDE, shell, and related workflows so teams can move faster without giving up judgment, safety, or accountability. For the free trial and platform installers (Bob IDE, Bob Shell), see Start a free trial.

Relationship to the product.
This book is written from practitioner experience; it is not product documentation and does not represent the IBM Bob product organization. Nothing stated here should be interpreted as a description of committed features, supported configurations, or IBM Bob's roadmap or future direction. If anything in this text conflicts with the shipped product or with IBM's official product materials, treat that mismatch as an editorial error. The product itself, along with IBM's authoritative product documentation and official IBM communications about IBM Bob, supersedes this book.

Personal use.
You may reproduce this publication for your personal, non-commercial use provided that all proprietary notices are preserved. However, you may neither distribute nor display this publication publicly, nor make derivative works of it, or any portion thereof, without the express consent of IBM.

Commercial use.
As a representative on behalf of an enterprise, you may reproduce, distribute, and display this publication solely within your enterprise provided that all proprietary notices are preserved. You may not make derivative works of this publication, or reproduce, distribute, or display this publication or any portion thereof outside your enterprise.

IBM legal notices.
For IBM website terms of use, DMCA and related notices, supply-chain statements where applicable, and other legal topics, see IBM Legal. For IBM copyright lines, trademark listings, fair-use guidance for IBM marks, and rules for referring to IBM product names, see Copyright and trademark information.

Preface

This book is not about teaching you how to use an AI tool.

It is about how software delivery changes when an agent can read the repo, propose a plan, execute bounded work, and leave behind evidence.

It is about understanding when and why Bob is useful, how to work with it without losing control, and how to integrate it into real development workflows where correctness, security, and long-term maintainability matter more than speed alone.

Bob exists because writing code is no longer the hard part. Designing systems, changing them safely, and keeping them valuable over time still is.

This book assumes you are part of software delivery.

You may write production code every day. You may work in QA, architecture, product, security, enablement, or engineering leadership. The common thread is that your decisions affect what software gets built, what changes are allowed, and what evidence proves the work is safe.

The structure reflects that broader scope.

Part I explains why the work changes. It starts with why Bob exists, then covers the transition nobody talks about, the operating loop this book uses throughout, and Bob’s own mental model.

Part II moves into the human operating loop. The IDE chapter frames the interface as a cockpit, not a developer test. The context chapter explains why most "hallucinations" are really context and constraint failures. The contracts chapter shows how to define done before execution. The first task chapter then walks through one accountable change end to end.

Part III covers safe change in real systems: review as evidence, modernization without behavior drift, security and permissions, and the explicit non-goals where judgment must stay human.

Part IV covers teams, roles, and governance. It looks at shared workflows, role-specific leverage, skills and rules as team memory, measurement that is not theater, mindset, and how to keep Bob boring enough to scale.

If you live in the IDE most days, Parts II and III are the steady-state loop. If you are

shaping process, risk, or rollout, Part IV is where the book becomes a field guide instead of a feature guide.

It does not try to convince you that AI is the future.

It shows you how to work with Bob in the present without handing off judgment.

Part I: Why the Work Changes

Chapter 1. Why Bob?

1.1. Code Is Cheap. Software Isn’t.

Writing code has never been easier. AI tools can spit out functions, classes, whole modules. They can turn a rough requirement into something that compiles. They can refactor. They can draft tests.

Code still is not software.

Software is what runs in production. It changes. More than one person has to understand it. Someone has to keep it alive after the person who wrote it leaves.

Code is text. Software is a system.

Systems are hard.

1.2. The Problem Bob Is Actually Solving

The usual pitch for AI coding assistants is speed: autocomplete, boilerplate, suggested implementations. That helps.

The bottleneck is elsewhere: knowing what to build, changing live systems without breaking them, and choosing approaches that do not turn into regret six months later.

Bob is aimed at that stack of problems.

Bob still writes code, but the point is systems thinking: what you are changing, why it is justified, and how you explain it to the people who own the outcome.

That is a different job than "generate the next function."

1.3. Why "AI Coding Assistants" Miss the Hard Parts

Most AI tools are strong at the mechanical layer:

- Generating code from descriptions
- Completing patterns you’ve started
- Translating between languages or frameworks
- Writing boilerplate and repetitive code

- Suggesting implementations for common problems

That work matters. It is also the comparatively easy slice of the job.

Where they tend to struggle:

- Understanding the intent behind a change
- Evaluating whether a change is safe
- Considering impact across multiple files
- Staying consistent with existing patterns
- Explaining why something should be done a certain way
- Helping you make decisions you can defend later

If you frame the job as code generation, these gaps stay invisible. In practice the job is decision-making under constraints.

The scarce part is rarely typing speed. It is deciding what deserves to exist in the codebase at all.

1.4. What Bob Treats as Non-Negotiable

Bob bakes in a few beliefs about what matters in software work. You cannot switch them off. That is deliberate.

Opinion 1: Intent matters more than implementation

Bob pushes on what you are trying to achieve, not only what you want typed. The goal is to surface the problem before you marry a solution.

Opinion 2: Evidence beats assumptions

Recommendations come from your repo as it exists: code, tests, config. Not from a guess about what might be there.

Opinion 3: Judgment stays with humans

Bob can surface findings, risks, and options. Bob does not replace your call on what to do, what is acceptable, or what ships.

Opinion 4: Auditability is not optional

Suggested changes tie back to reasons you can trace. That supports compliance when you need it; more broadly, it is how software stays governable over years.

Opinion 5: Boring is better than clever

Simple shapes and obvious patterns beat cleverness when someone else has to change the code in a hurry. Boring is often easier to evolve.

Those constraints are why Bob feels different from a generic completion engine—and why Bob is aimed at production work, not demos.

1.5. When Bob Helps, and When It Shouldn’t Be Used

Bob is not the right hammer for every nail.

Bob helps when:

- You’re changing existing systems and need to understand the impact
- You’re reviewing code and want to catch issues before they reach production
- You’re modernizing legacy code and need to preserve behavior
- You’re working in regulated environments where decisions need to be documented
- You’re onboarding new team members who need to understand the codebase
- You’re making architectural decisions that affect multiple components

Reach for something else when:

- You want raw generation speed and little reflection
- You’re prototyping and do not care yet whether the code survives
- The code is disposable and will never see production
- You want the tool to decide what ships
- You want magic with no curiosity about how it works

Bob targets production software: systems that have to run, change, and be explained to people who were not in the room when the decision happened.

Quick experiments belong on lighter tools. Long-lived systems are where this design

pays off.

1.6. What This Means for You

The book is about mindset for AI-assisted development, not a feature tour.

You will work on stating intent clearly, weighing evidence from the repo, and making calls you can defend in a review or an audit.

This book uses Bob as the hands-on layer. The habits are what transfer.

By the end, you should have a clearer sense of when automation earns its place, how to stay responsible for outcomes, and how to keep systems changeable without gambling production.

Continue with The transition nobody talks about—before the mechanics make sense, the human shift has to be visible.

Chapter 2. The Transition Nobody Talks About

2.1. When the Unit of Work Changes

The first strange thing about agentic development is not the model.

It is the feeling that your normal instincts only solve half the job.

In a traditional workflow, the unit of work often feels concrete: open files, change code, run tests, commit. In an agentic workflow, the unit of work expands before any code changes happen. You are framing intent, naming constraints, feeding context, reviewing a plan, checking evidence, and deciding what Bob is allowed to do next.

That shift is where many people wobble.

The tool feels slower. The workflow feels more verbal. The pauses can feel like friction. They are not accidents. They are what happens when software work stops being only about implementation and becomes partly about steering.

2.2. Traditional Habits vs. Agentic Habits

Traditional development rewards fast implementation.

Agentic development rewards clear framing.

Traditional IDE work starts with files.

Agentic work starts with intent.

Traditional automation is deterministic: a build, a script, a pipeline step. You know in advance what the mechanism will do if the inputs stay the same.

Agentic work is probabilistic. Bob can reason well, read evidence well, and still take a path you would not have chosen first. That is why planning, approval, tests, and review boundaries matter more here than they do with plain scripting.

Traditional documentation often arrives after the work.

Agentic work needs lightweight evidence while the work happens: a plan in the task, preserved behavior called out in the prompt, a note in the pull request about what Bob reviewed, what it changed, and what was verified.

2.3. Why It Feels Slower Before It Feels Better

Many teams initially judge agentic work by the wrong benchmark.

They compare "one person typing directly into a file" with "one person describing the job, waiting for analysis, then reviewing a plan." Of course the second path can feel slower in the first minutes.

The comparison changes when the task is real:

- The code is unfamiliar.
- The behavior is only partly documented.
- Several files may change.
- Constraints are spread across tickets, tests, rules, and team habits.
- The work needs to survive review by someone who did not watch the chat.

In that environment, the extra time spent up front is not overhead. It is the cost of making intent and evidence visible before changes become expensive.

2.4. Probabilistic Systems Need Contracts

This is the uncomfortable middle between "AI wrote code" and "the team shipped software."

Because Bob is not a deterministic compiler pass, you need contracts around the task:

- What outcome are we trying to achieve?
- What must not change?
- Which files or systems are in scope?
- What evidence proves the result is correct?
- When should the work stop instead of continuing?

For a feature, the contract might be acceptance criteria.

For a bug, the contract might be a failing test that turns green.

For modernization, the contract is often preserved behavior.

For documentation, the contract is accuracy against shipped behavior, not just plausible wording.

The contract does not tell Bob exactly how to write the code. It tells the team how to recognize a correct outcome.

2.5. Evidence Has to Travel With the Work

In agentic development, explanation is not ceremony added at the end.

It is part of keeping the work reviewable.

If Bob explored three services before suggesting a change, that matters.

If Bob found hidden dependencies, that matters.

If Bob proposed a risky shortcut and you rejected it, that matters too.

This is why good teams leave a trail:

- the original task intent
- the approved plan
- the verification steps
- the preserved constraints
- the final explanation in a PR, ticket, ADR, or task history

Without that trail, the code lands but the judgment disappears.

2.6. What Changes for Different Roles

This transition is not only for developers.

Product managers help make intent testable.

Architects help turn constraints into executable rules.

Testers and QA help define the behavior contract before changes start.

Security and compliance help shape permissions, trust boundaries, and evidence retention.

Managers help decide which habits are visible, repeatable, and safe enough to scale across a team.

That is why this book needs to read like a migration guide for people, not a feature tour for one persona.

2.7. What You Learned

The first weeks with Bob feel strange because the unit of work changes. You are not only writing code. You are steering intent, evidence, permissions, and verification. The extra structure is not bureaucracy. It is how probabilistic work becomes accountable.

The Agentic SDLC turns that shift into one loop the rest of the book can keep reusing.

Chapter 3. The Agentic SDLC

3.1. A Loop You Can Remember

If readers remember only one model from this book, it should be this one:

> *Orient → Bound → Plan → Act → Verify → Explain*

It is simple on purpose.

Developers can use it.

Architects can use it.

Product managers, testers, security reviewers, and managers can all use it without pretending they live in the IDE full time.

The loop does not make software easy. It makes the work legible.

3.2. Orient

Orient means understanding the real system before asking for change.

That includes:

- the business goal
- the code and configuration already in place
- the existing behavior
- the tests and failure modes
- the surrounding team context

This is the stage where Bob reads the repo, related files, tests, and history instead of improvising from a vague prompt.

It is also where humans resist the urge to skip straight to implementation.

3.3. Bound

Bound means deciding what is allowed to happen and what is not.

This is where intent becomes constrained enough to be safe:

- in-scope files and systems
- out-of-scope changes
- required approvals
- environment limits
- security and compliance restrictions
- stop conditions

Boundaries matter because confidence is not the same thing as authorization. Bob may have an idea; that does not mean the task deserves broader permissions.

3.4. Plan

Plan means Bob proposes a path before changing anything.

A useful plan names:

- what will change
- why it will change
- what might break
- what will stay unchanged
- how the result will be verified

The plan is where probabilistic reasoning becomes discussable. It is the checkpoint where teams catch missing context before a diff exists.

3.5. Act

Act means executing only the approved slice.

This is intentionally smaller than "do everything."

Good agentic work narrows action:

- one refactoring phase

- one review pass
- one bounded documentation update
- one approved set of files

The point is not to keep Bob timid. The point is to keep action proportional to the evidence and the permissions granted.

3.6. Verify

Verify means checking behavior, not admiring motion.

Verification can include:

- tests
- diff review
- manual inspection
- security review
- preserved behavior checks
- acceptance criteria

This is where many bad AI stories are born. The code looked plausible, the tests looked green enough, and nobody checked whether the original contract was still true.

3.7. Explain

Explain means leaving behind evidence someone else can review later.

That record may live in:

- the task history
- a pull request
- a ticket
- a short ADR
- release notes

Explanation is not for vanity. It is how the next person understands what Bob did, what humans approved, what risks were acknowledged, and what was actually verified.

3.8. Inner Loop, Outer Loop, Evidence Ledger

The loop runs at more than one scale.

The inner loop is the immediate task: understand, constrain, plan, execute, verify, explain.

The outer loop is the team system around it: shared rules, modes, security controls, review habits, and measurement.

The evidence ledger is what connects them. Plans, findings, task history, diffs, and verification notes let one task become organizational learning instead of isolated chat output.

3.9. Why This Model Helps Mixed-Role Teams

Non-developers do not need to become full-time coders to matter in this loop.

They contribute at different points:

- Product defines intent and non-goals.
- Architecture defines constraints and trade-offs.
- QA defines behavior expectations and edge cases.
- Security defines permissions and trust boundaries.
- Engineering leads define the rollout posture and review expectations.

The IDE may contain the artifacts, but the job is larger than the IDE.

3.10. What You Learned

Orient, Bound, Plan, Act, Verify, Explain is the book's canonical operating loop. It turns agentic work into something teams can review, teach, and govern instead of treating it like clever autocomplete with better marketing.

How Bob thinks explains why Bob's internal model aligns with that loop: intent, evidence, and judgment stay separate on purpose.

Chapter 4. How Bob Thinks

4.1. Understanding Bob's Model of Software

A lot of AI tooling treats the repo as text: patterns in, diff out.

Bob treats it as a system with intent—why this module exists, what a change is supposed to fix, what breaks if you guess wrong.

That stance changes the workflow.

When you ask Bob for help, it does not jump straight to code. It pins down what you want, reads what is already there, and only then thinks about what could break. That costs more time up front than a raw generator; it usually saves you later.

The rest of this chapter is that model in plain language.

4.2. Intent, Evidence, Judgment

Bob's mental model has three parts.

Intent, evidence, and judgment stay separate on purpose. Mixing them is how you get confident code that solves the wrong problem.

Intent: What you're trying to achieve

Intent is the goal. The reason for the change. The problem you're solving.

Bob always starts with what you want to accomplish—not which file to touch first. That alone forces clarity: "fix the service" without a symptom is useless, and "improve performance" needs a number or a scenario. Good intent is specific enough to check later.

Evidence: What actually exists

Evidence is the current state of the system. The actual code. The actual tests. The actual configuration.

Bob does not guess what your repo looks like—it reads it. Before it suggests a change, it walks files, dependencies, tests, and current behavior. That is slower than pattern-matching from training data, and it is how recommendations stay tied to **your** system.

Judgment: What should be done

Judgment is the decision: which option, which trade-off, what you accept if something goes sideways.

That call stays with you.

Bob can list findings, explain risks, and sketch options. It cannot decide what is acceptable for your team, your schedule, or your risk appetite. You own what ships; Bob supplies the packet of evidence to justify or challenge it.

4.3. Why Bob Separates Planning from Execution

Bob works in two phases.

First, Bob plans. Then, Bob executes.

The split is deliberate: you get a reviewable plan before anything lands in the repo.

The planning phase

During planning, Bob:

- Understands your intent
- Examines the evidence
- Identifies what needs to change
- Explains the risks
- Suggests approaches
- Asks for your approval

Nothing happens during planning: no file edits, no generated code, no decision locked in.

Planning is where you build shared understanding of what should happen before anyone edits files.

The execution phase

During execution, Bob:

- Makes the changes you approved

- Follows the plan exactly
- Reports what was done
- Highlights any issues
- Asks for verification

Execution should be mechanical: predictable, boring.

Surprises belong in planning, while you can still change your mind—not halfway through a merge.

Why this matters

Plenty of tools interleave planning and execution: they emit code while the goal is still fuzzy, or edit files while the failure modes are still unclear. That feels fast until it isn't.

Bob splits the phases on purpose. Software change works better when you know what will happen before it happens, and when execution starts only after the plan has a human thumbs-up.

You trade raw speed for fewer nasty surprises. For production systems, that trade is usually worth it.

4.4. Constraints Over Creativity

Bob is not there to invent clever one-offs. It stays inside your patterns, tests, and policies—the constraints it infers from the code you already shipped.

What are constraints?

Constraints are the rules that make your system work. The patterns you follow. The conventions you maintain. The boundaries you don't cross.

Constraints include:

- Your coding standards
- Your architectural patterns
- Your testing requirements
- Your security policies

- Your team conventions

Bob infers those constraints from what you already shipped and from the rules you set for it. Classes, error paths, test style—the patterns already in the repo.

Suggested changes are supposed to look like your team wrote them.

Why constraints matter

Creativity in application code is usually a liability: surprises for the reader, broken expectations, harder maintenance.

The goal is consistency, not novelty—a change that reads like your team wrote it, follows the same patterns, and matches the house style.

Predictable output is easier to review and easier to rip out later. Boring is good.

When constraints conflict

Sometimes constraints conflict. The right fix breaks a local pattern, or the exception is worth documenting.

Bob does not pick the winner. It flags the tension—typically something like: this change solves the problem, but it breaks an existing pattern; here are the ways out.

You decide whether the pattern bends, whether the exception is justified, and what matters more for this release.

Bob surfaces the trade-off; you pick.

4.5. Why Bob Asks Before It Acts

Bob does not edit the repo without a checkpoint.

Obvious, maybe—but the checkpoint does real work.

Asking forces clarity

Approval means Bob has to say what will happen, what edits that implies, and why that approach beats the alternatives. If the explanation is muddy, treat that as a signal: fix the plan before you touch production paths.

The pause is the safety rail—time to notice the wrong abstraction before it lands.

Asking creates accountability

When you approve, you own the outcome. The plan was yours to accept; the risks were yours to weigh.

That keeps control and responsibility on the human side of the keyboard. Bob is a tool; the decision still sits with you.

Asking enables learning

Each approval round shows you how Bob prioritizes: what it surfaces as risk, what it treats as evidence, where it asks you to choose.

Over time that trains tighter intent, sharper reads on the repo, and faster judgment calls. Task throughput is a side effect; clearer thinking is the point.

When asking becomes automatic

After enough reps, the interruptions feel less like friction and more like a habit—a forced review before the irreversible part.

That is often where Bob earns its keep: less as a typist, more as a thinking partner for what should be typed at all.

4.6. What This Means for Using Bob

If you accept this model, you use Bob for better decisions first and keystrokes second.

Let it handle the mechanical stretch—reading the repo, drafting edits against an approved plan—while you spend attention on intent, trade-offs, and what you are willing to ship.

The IDE shell is not a developer test maps the same ideas to panels and task phases without turning into a feature infomercial.

Bob is neither magic nor autopilot. The loop is systems thinking, evidence from what exists, and human judgment you can explain later. Everything else is implementation detail.

Part II: The Human Operating Loop

Chapter 5. The IDE Shell Is Not a Developer Test

5.1. A Quick Tour Without the Noise

Treat this as orientation for the shell—not a panel inventory, not a stealth quiz on whether you count as technical.

Bob sits in an IDE because the work touches real artifacts: code, tests, configs, tickets, docs, diffs, task history. That does not mean every reader needs to live like a full-time coder.

For many roles, the IDE reads more like a cockpit than a craft bench: it surfaces the system, the evidence, and where approval happens. Nobody needs to ship production Java to participate; people need a shared place to inspect those artifacts and change them without guessing.

You need the handful of surfaces that turn intent, evidence, and judgment into actions you can take in the UI. Shortcuts are optional.

Below is how that maps in practice.

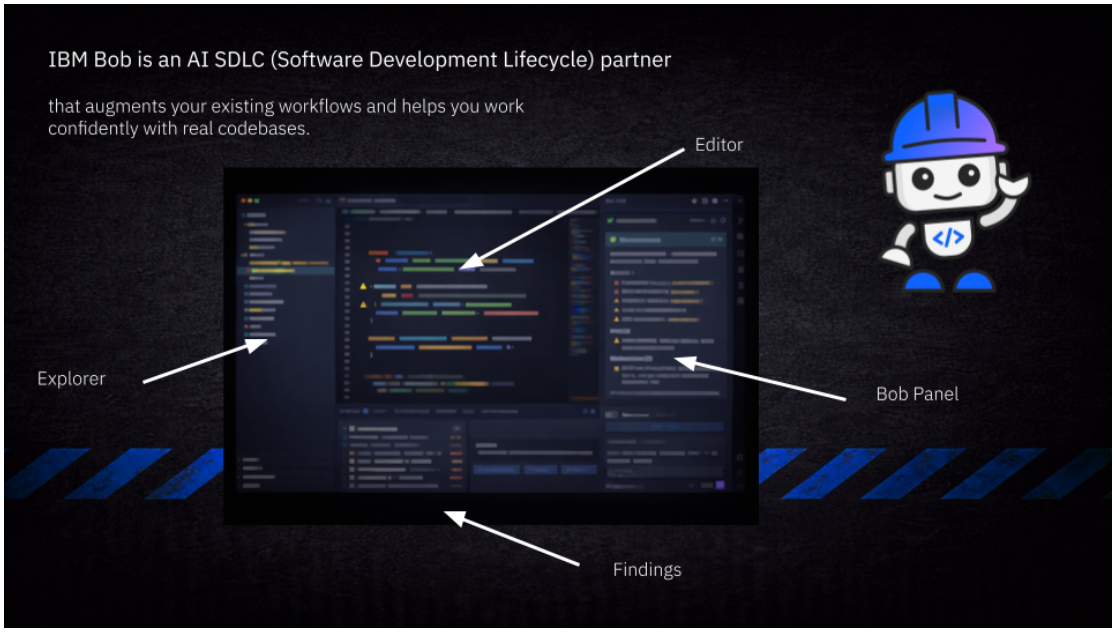


Figure 1. [the user interface]

The layout stays sparse on purpose.

Bob is not trying to be a second IDE inside your IDE. You get the pieces that line up with intent, evidence, and judgment—not a wall of optional chrome.

What follows is that layout in order.

5.2. The Bob Side Panel

The side panel is your working surface: tasks, plans, approvals.

In product terms, a **task** is one chat thread: checkpoints and history stay on that thread, so start a new task when the topic changes (one chat, one goal).

The side panel has two main sections:

Task input

At the bottom, you state intent: what should be true when this work is done.

Keep it goal-shaped—symptoms, constraints, success checks—not the patch you think you need.

For example:

- "Add validation to the user registration endpoint"
- "Update the database schema to support soft deletes"
- "Refactor the payment service to use the new API"

Bob uses that text to anchor planning—what you want to achieve before anyone proposes diffs.

Current task status

Above the input, you see the current task status. This shows what phase Bob is in.

The phases are:

- Understanding: Bob is reading your intent and examining the code
- Planning: Bob is creating a plan based on evidence
- Waiting for approval: Bob has a plan and needs your decision
- Executing: Bob is making the approved changes
- Complete: Bob has finished and is showing results

Phase and next step stay visible in the status block—you should not have to guess whether Bob is still reading the repo or waiting on you.

Type a slash (/) in the chat box to open **slash commands**—quick starters such as review workflows, plus any commands your team added under project or user command folders (see Slash commands in IBM Bob documentation).

Task history

If you close a specific task, you see an overview of your previous tasks. This is your audit trail.

Search and export when you need to reconstruct what was asked, what was proposed, and what landed—handy when someone asks six months later why a decision looked reasonable at the time.

5.3. Tasks, Plans, and Findings

Bob organizes work into three types of information—the same intent / evidence / judgment split from How Bob thinks, now split across UI buckets instead of paragraphs.

Tasks

A task is what you want to accomplish—the intent.

Strong tasks read like an outcome you could verify: scope, failure mode, done-when.

Good task: "Add input validation to prevent SQL injection in the search endpoint"

Bad task: "Make the code better"

Bob pushes vague asks toward something testable and asks questions when the goal is still fuzzy.

Plans

A plan is Bob’s proposal for how to accomplish the task. The evidence-based approach.

Plans include:

- What files will be changed
- What the changes will do

- Why these changes are necessary
- What risks exist
- What alternatives were considered

A plan should be concrete enough to approve or reject without squinting.

During planning you see descriptions of what will change and why—not raw code on purpose, because syntax pulls attention away from whether the approach is right.

Findings

Findings are issues Bob discovers while examining your code. The evidence.

Findings include:

- Security vulnerabilities
- Performance problems
- Inconsistencies with patterns
- Missing tests
- Configuration issues

Each finding has:

- A description of the issue
- The location in the code
- The severity level
- Suggested fixes
- Explanation of the risk

Findings do not automatically become tasks. You choose what is worth acting on; Bob does not score your priorities for you.

5.4. How Bob Reads Your Project

Bob does not only read the buffer you have focused—it walks the project shape, dependencies, config, tests, prevailing code patterns, and whatever docs you already

wrote. Each slice answers a practical question (where a change belongs, what versions you are pinned to, what behavior is already locked by tests, what style the team actually uses). Only after that pass does it propose diffs, which is why the planning phase can feel quiet for a few seconds.

Built-in modes—Ask, Plan, Code, Advanced, Orchestrator—ship with every install and cap what Bob may do in a thread. Custom modes your organization distributes sit beside built-ins in the same dropdown when your admin enables them. Day-to-day discovery runs through Settings and the slash menu—Modes in IBM Bob documentation is the product reference.

5.5. What Bob Never Touches Unless You Approve It

Boundaries first.

Bob does not make changes without your approval. You can widen permission with per-tool approvals or global tool approvals; **auto-approve** skips those prompts. That is faster until it is not—mistakes scale with access—so many teams keep explicit prompts for anything that touches files or commands.

Before file changes, Bob can save a **checkpoint** (a restore point for that task). **Restore files only** rolls files back and keeps the chat; **restore files and task** also trims later messages when the thread went wrong. Checkpoints complement Git—they do not replace commits.

5.6. The Interface Reflects the Model

The side panel lines up tasks, plans, and findings with the model above. Phases spell out planning-before-execution, and approval gates sit where you can see them—the layout aims for obvious, not clever. You should always see where Bob is in the flow and what still needs a human yes.

Spend a minute clicking phase, approval, and history so the muscle memory matches the mental model. Then Context is the work explains what Bob should see, Contracts Before Code defines what the task must satisfy, and Your first accountable task runs the whole loop end to end.

Go deeper in IBM Bob documentation:

- Modes

- Skills

Chapter 6. Context Is the Work

6.1. Why Context Is a Budget, Not a Dump

Bob's answers ride on the **active context**: chat history, what you attach with @, tool output, and the product's own instructions and rules. That pile is always capped.

Everything in view has to fit one **finite context window**—how much text Bob can load into working memory for this turn. Long threads, giant pastes, and greedy folder mentions fight for the same slots. When the window turns noisy or stale, output quality drops: confident wrong answers, pointless tool churn, fixes that wander away from what is actually in the repo.

This chapter covers how to point Bob at the truth, trim payload, recover when context goes bad, and keep secrets out of routine reads. For official detail, see context window management and context mentions in IBM Bob documentation.

6.2. How the Window Stacks

When Bob answers, the product blends layers like this:

- **System and rules** — safety, behavior, and standing instructions.
- **Chat history** — what you and Bob already said in this task.
- **Context mentions** — files, folders, git state, URLs, problems, terminal output you attached with @.
- **Tool output** — search hits, command logs, fetched pages, MCP results.

All of it draws on **one** budget. If the IDE shows token pressure, treat it like a fuel gauge: shrink scope or open a clean task.

Habits that help

- Prefer **small, relevant slices**—mention a file or line range instead of a whole tree when you can.
- **Split topics across chats**—a new task is often cheaper than dragging yesterday's tangent forward (see The IDE shell is not a developer test).
- **Watch usage**—heavy attachments and verbose tool replies add up fast.

Three constraints worth internalizing:

- **Finite working memory.** The window is not the whole repository; it is the current working set.
- **Recency and relevance compete.** New text can crowd out older facts that still matter.
- **Input quality dominates output quality.** Strong evidence beats clever prompting.

6.3. Signal Versus Noise

Two requests with the same intent can land very differently depending on what sits in the window. Signal is concrete, verifiable, and scoped to the job. Noise is broad, stale, or mixed-topic.

High signal input	Low signal input
Exact file snippet and explicit goal	Paraphrased code and vague intent
Actual stack trace or terminal output	Memory of an error “somewhere in utils”
One mission per thread	Several unrelated goals in one chat
Current docs or specs for the API you mean	Assumptions from outdated free-form recall

Curating evidence matters more than stuffing more text into the chat.

6.4. Point Bob at the Truth: context mentions

Typing @ attaches workspace artifacts on purpose. That usually beats mangled paths or screenshots of code.

A usable default recipe: precise @ mentions, exact error text (or @terminal / @problems), and a one-sentence goal.

- @/path/file — You need exact file contents; on large files, use a line range—for example @/path/File.java:80-120 when only part of the file matters.
- @/path/folder — Several sibling files matter; folder mentions are **non-recursive**—do not assume deep trees.

- @problems — Diagnostics from the Problems panel should ground the answer.
- @terminal — Build or CLI output is the evidence—avoid manual copy-paste when this is available.
- @git-changes or a commit hash — You are reasoning about diffs, commits, or uncommitted work.
- @ plus a URL — External docs or specs should anchor the reply.

Ignore rules and deliberate mentions

Explicit file and folder @ mentions **bypass** `.bobignore` and `.gitignore` when fetching text—so you can pull a path in on purpose. Watch what you attach; convenience cuts both ways.

`.bobignore` keeps secrets, keys, and noise out of **routine** tool reads. Version a shared ignore policy like anything else you review: explicit, in repo, owned. See `.bobignore` in IBM Bob documentation for configuration detail.

Good context engineering combines **what you include** with **what stays out by default** —`.bobignore` is a boundary tool, not a substitute for thinking before you attach.

6.5. Grounding Habits

- **Do** mention exact files and line ranges when you can.
- **Don't** paraphrase code from memory when @ can fetch the real text.
- **Do** paste real stack traces or use @terminal / @problems.
- **Don't** describe errors vaguely when the panel or terminal already has the evidence.
- **Do** keep one clear outcome per task thread.
- **Don't** let unrelated goals share the same chat when a fresh task costs less than confusion.
- **Do** link authoritative docs or specs when the answer must match an external contract.
- **Don't** treat earlier chat turns like durable memory across sessions—re-ground with mentions when it matters.

6.6. Big Repos and Heavy Files

Huge repos punish vague missions. Work **narrow**: one service, one package, one failure mode. Say what matters in a sentence, link the ticket, attach only files that actually flip the answer. For moving across big trees without drowning the window, see working with large projects in IBM Bob documentation.

6.7. Large spreadsheets, documents, and slides

Office files look small on disk and still explode in the window: `.xlsx`, `.docx`, and `.pptx` drag sheets, revision metadata, slide masters, speaker notes, hidden ranges—the same working memory that holds code has to swallow that bulk before your question gets attention.

Treat context like desk space, not infinite cloud. If you would not ask a human to read the whole workbook before lunch for one question, do not drop the whole file on Bob.

What helps in practice:

- **Spreadsheets** — Filter to the rows that matter, hide unused sheets, or copy one sheet to a smaller file. Export a bounded CSV or paste the header row plus twenty representative rows instead of the full grid.
- **Documents** — Paste the paragraph or section you mean, or export a short excerpt. Long policy PDFs rarely need every page for a wording question.
- **Slides** — Trim speaker notes if they are noisy; export key slides as images only when visuals are the point.

When only a script can slice the attachment cleanly, hand it to whoever owns automation on your team instead of assuming every reader runs glue code.

For habits that keep large Office attachments and mentions bounded, see context mentions and context window management in IBM Bob documentation.

6.8. Rough Size: What “200k Tokens” Means

IBM Bob’s context window is on the order of **two hundred thousand tokens**—large, still shared. A **token** is a small chunk of text (part of a word, punctuation, or code). For English prose, plan on **about four characters per token** so you get an intuition for “book-sized” limits—then remember chat history, @ attachments, and tool output all draw from the same pool.

Some capacity is **reserved for the product itself**—on the order of tens of thousands of tokens for instructions and safety behavior. What is left is what you spend on the task. Nothing stays loaded like a human memory of files; each turn re-ingests what you put in the window.

Very long threads add another wrinkle: the product may **condense** or summarize older turns to keep the conversation moving. Momentum goes up; verbatim detail from early messages can soften. When precision matters, prefer a clean task plus a tight handoff note over an endless thread.

Rough footprints (order of magnitude)

These numbers move with model and product releases; use them to compare **relative** weight, not as guarantees:

- Reserved system and tool definitions: on the order of tens of thousands of tokens (often cited around **12–15K** in field notes).
- A single mid-sized source file: a few thousand tokens.
- A whole module folder: tens of thousands of tokens—often cited around **30–50K** depending on size.
- Office exports and giant logs: tens of thousands of tokens fast.
- Long chat history alone: can reach tens of thousands to one hundred thousand tokens or more when left unchecked.
- Auto-condense thresholds (when the product summarizes history): often cited around **140K** in practitioner notes; hard caps are commonly discussed around **200K** for IBM Bob—same ballpark as this chapter’s headline window size.

Token pressure is structural: it affects answer quality, latency, and—when usage is metered—cost.

Why it runs out

Long threads, huge copy-pastes, wide folder mentions, and verbose tool replies compound. When the budget is tight, start a **fresh chat**, mention **narrow paths** or line ranges, and paste **only the error or paragraph** you need—not the whole log.

6.9. A Minimal Grounding Loop

When quality matters more than raw speed:

- **Define** — one clear outcome for this task.
- **Ground** — only evidence that changes the answer (@, terminal, docs).
- **Verify** — outputs against sources, not against confidence.
- **Reset** — when the topic or assumptions drift.

Escalation rule: if a **second correction** still misses, treat it as context drift—restart with a smaller, cleaner context set instead of arguing in place.

6.10. When Context Goes Bad: Poisoning

Context poisoning is wrong, misleading, or irrelevant material sitting in the active window long enough to steer later turns. See also context poisoning in IBM Bob documentation.

Symptoms

- Answers drift away from the real goal.
- One bad assumption gets repeated louder in later turns.
- Confident references to symbols or APIs that are not in your repo or docs—**phantom certainty**.

Also watch for the usual quality collapse: confident wrong answers, tool calls that chase ghosts, fixes that drift from the repo.

First aid

Strip bad tool output when you can, tighten @ scope, restate the goal in one sentence.

Escalate by severity

- **Slightly off-track** — restate the goal; add the missing @ mentions.
- **Wrong file or method** — remove incorrect mentions; narrow scope.
- **Confident but imaginary API** — ground with authoritative docs (@ URL or fetched spec).

- **Mixed topics** — split: new task with a short handoff note only.
- **Deep poisoning** — hard reset: new task, minimal clean context.

The reliable fix is usually not “argue longer”; it is **reset context and re-ground**.

Hard reset

Often the fix is a **new chat** with a tiny, honest context set.

6.11. When Windows Feel “Heavy”

Practitioners sometimes talk about **smart zone** versus **dumb zone**: lower pressure on the context window versus high pressure where reasoning gets brittle. The boundaries are not a formal product threshold—they move with model, task, and prompt shape—but the mental model is useful.

Common failure patterns under load:

- **Lost in the middle** — models often attend strongly to the start and end of a stuffed prompt; facts buried in the center suffer first.
- **Context rot** — huge mixed windows (docs, logs, tools, old chat) raise noise and weaken multi-step reasoning.
- **Instruction pressure** — as context grows, following every constraint gets harder, especially when instructions stack.

If constraints suddenly slip, logic regresses mid-session, or answers wander after a long thread, treat the window shape as the prime suspect—then trim, hand off, or restart.

6.12. Most Hallucinations Are Context Debt

Not every wrong answer is random model failure.

Many bad outputs start earlier:

- a missing constraint
- a missing source of truth
- a missing dependency
- a missing environment detail

- a missing test expectation
- a business rule that never entered the task

The failure chain is dull and expensive: ambiguity hides inside assumptions, assumptions harden into plans, plans become edits, edits become defects.

Calling that chain "hallucination" gives it too much mystery. Most of it is context debt nobody cleared before asking for action.

6.13. The Context Pack

Drop the fantasy of "the whole repo in one chat." Think in **context packs** instead.

A context pack is the smallest bounded evidence Bob needs for one task:

- the task intent
- the relevant files
- the known constraints
- the tests or expected behavior
- the related ticket or customer scenario
- the non-goals
- the stop condition

Ship a packet another reviewer could follow—not a brain dump.

That is why context work is not only for developers. PMs narrow intent. QA pins verification. Architects spell constraints. Developers attach the right code and tests.

6.14. Session Handoff and Long Threads

Before you switch topics or abandon a long thread, capture a small handoff artifact—often a Markdown file in `docs/`—and use it as the anchor for the next task. Keeps decisions durable without dragging the entire chat forward.

```
Before we switch topics, create docs/handoff-[feature].md:

## Goal
## Decisions
```

```
## Files touched
## Open questions

Use only what we agreed.
Stop. Do not implement yet.
```

Other patterns teams reuse:

- **Session handoff** — goals, decisions, touched files, open questions.
- **ADR stub** — constraints, chosen approach, rejected alternatives.
- **Repro / runbook** — exact commands, expected versus actual, minimum @ set.

Teams also reduce load by **compacting** state into a short note and continuing in a clean task, splitting research from implementation across sessions, or isolating broad exploration in a helper flow so the main thread only receives short findings—tactics vary by org; the pattern is **stop paying window rent for work you no longer need in view**.

For a practitioner-oriented walkthrough of subagents and context limits—adjacent to how teams isolate exploration—see Exploring Subagents and Context Limits (video).

If your install exposes helper agents or split flows for exploration, keep the main task focused on short, cited findings—see modes and task behavior in IBM Bob documentation for what your organization enabled.

6.15. Docs MCP server (optional)

arabold/docs-mcp-server indexes documentation sources so answers can ground on **current** versions—sites, repos, packages, local folders—instead of only static training priors. Typical use: index the stacks you ship with (for example Quarkus, Jakarta EE, internal portals your admin allows), then ask Bob to consult that MCP source before recommending an approach.

Configure MCP servers only through paths your organization allows; treat indexed documentation like any other connector—scoped, reviewed, and aligned with what production runs. See Security, permissions, and MCP trust and IBM Bob documentation for wiring detail.

6.16. Healthy Chat Checklist

- **Before** — one goal per task, direct @ where possible, exact errors from Problems or

terminal.

- **During** — watch token pressure, review iteratively, keep folder scope tight.
- **Before you leave** — write a handoff note (and commit it when your team tracks decisions in Git).
- **Hard no** — no secrets in chat, no huge raw Office dumps, no pretending yesterday's chat is permanent memory.

6.17. Safety Habits That Pair With Context

- **No secrets in chat** — paste policies, not API keys; lean on environment files and ignore rules.
- **Know what auto-runs** — auto-approve and permissive MCP setups can turn a typo into an action; hold approvals until you trust the wiring (see Security, permissions, and MCP trust).
- **Readable output** — structured text survives reviews and audits better than a wall of noise.

For rollout, threat models, and MCP governance at enterprise depth, the same themes appear in IBM Bob security guidance.

Go deeper in IBM Bob documentation:

- Context mentions
- Context window management
- Security guidance

6.18. What You Learned

Context is shared budget: history, mentions, and tools compete for the same window. Small @ slices, fresh tasks when the topic shifts, and trimmed attachments keep answers tied to the repo. `.bobignore` filters routine reads; explicit @ overrides when you intend to pull a path through.

Contracts Before Code turns this evidence into a bounded definition of done. After that, Your first accountable task shows the same loop under real friction.

Chapter 7. Contracts Before Code

7.1. The Durable Thing Is Not the Exact Diff

In AI-assisted development, the exact implementation is less durable than many teams expect.

Bob may choose a different but still reasonable path depending on the repository state, the available rules, the mode, the tests, or the surrounding context. That means the lasting value is not "type these exact lines." The lasting value is the contract around the work.

The contract tells the team how to recognize a correct result even if the implementation details vary.

7.2. Contracts Beat Copy-Paste

Copy-paste tutorials assume one best path and one expected output.

Agentic work does not behave that neatly.

What you want before Bob edits code is not a frozen implementation. You want a bounded agreement:

- what outcome matters
- what must stay unchanged
- what evidence proves success
- what risk level is acceptable
- what should stop the task before it drifts

That agreement keeps the work reviewable even when the implementation is not identical from one run to the next.

7.3. What a Contract Looks Like

A useful contract is short enough to survive real work and concrete enough to verify later.

It usually includes:

- the task intent
- the success condition
- the non-goals
- the preserved constraints
- the verification method
- the stop condition

This is where the **context pack** from Context is the work becomes practical. Context gathers the evidence. The contract states what that evidence is supposed to protect.

7.4. Different Tasks Need Different Contracts

The shape changes with the job.

For a feature

The contract is usually acceptance criteria:

- what the user can do after the change
- what must not regress
- which behaviors count as done

For a bug

The contract is often a failing test that turns green, plus the guarantee that adjacent behavior stays intact.

For modernization

The contract is preserved behavior. Existing tests should still pass, public interfaces may stay fixed, and any intentional behavior change should be named instead of smuggled in.

For security work

The contract is not just "fix the warning." It is threat reduction plus no new exposure.

For documentation

The contract is accuracy against shipped behavior, not wording that merely sounds plausible.

7.5. Write the Contract Before Bob Acts

The right moment to define the contract is before execution, not after a good-looking diff appears.

That does not mean weeks of ceremony. It means one moment of discipline:

- What are we trying to achieve?
- What must remain true?
- How will we verify it?
- What would make us stop?

If those answers are fuzzy, the task is still under-specified.

7.6. A Small Example

Instead of this:

> *"Modernize AuthenticationService."*

Write this:

> *"Modernize AuthenticationService to Java 21 style while preserving all existing login, token refresh, and audit behavior. No public API changes. All existing tests must pass unchanged. Stop if the refactor requires a behavior change we cannot explain in the PR."*

That is not a perfect specification.

It is enough to steer planning, review, and verification.

7.7. Contracts Make Reviews Faster

A clear contract helps more than Bob.

It helps reviewers ask better questions:

- Did the change satisfy the stated outcome?
- Did the preserved behaviors stay preserved?
- Was the verification strong enough for the stated risk?
- Did the diff exceed the approved scope?

Without a contract, reviews drift into taste and guesswork.

7.8. What You Learned

The durable thing in agentic work is not the exact code Bob happened to write. It is the contract around the change. Define success, constraints, verification, and stop conditions before execution so the diff can be judged against something real.

Your first accountable task puts that discipline into one end-to-end example.

Chapter 8. Your First Accountable Task

8.1. From Prompt to Judgment

This chapter walks through one accountable task end to end.

It is not a sanitized demo. Messy choices stay in on purpose.

Speed is not the lesson. Reading Bob's output, stress-testing its recommendations, and keeping decisions you can explain—that is the lesson.

Context is the work narrowed what Bob should see. Contracts Before Code framed what success means. Here you watch both collide in a single flow.

The walkthrough uses a common starter problem: add validation to an API endpoint.

8.2. Defining Intent Clearly

Open an example Quarkus Java project like you can scaffold with this guide: https://quarkus.io/get-started/

Open the project with Bob and go to the side panel and type:

"Add validation to the greeting endpoint"

Bob responds:

> *I'll help you add validation to the greeting endpoint. Let me first examine the current implementation to understand what needs to be validated.*
>
> - *What type of validation would you like to add to the greeting endpoint? I can see the GreetingResource.java file is open. Please share what validation rules you need.*

Bob shows a few options to choose from: * Validate that the name parameter is not null or empty * Validate name length (e.g. min 2. max 50 characters) and format (only letters and spaces) * Add comprehensive validation: non-null, length constraints, character restrictions, and sanitization * Show me the current endpoint code so I can suggest appropriate validation.

That first reply is Bob refusing to invent scope. Pick an option and the intent tightens.

Bob responds:

> *Perfect! Here is the plan.*

Now the ask is specific enough to plan against—not magic wording, a bounded task.

The request lines up with what Contracts Before Code keeps asking for: a clear target, a visible outcome, and enough detail that Bob can read the repo instead of guessing.

8.3. Letting Bob Explore the Codebase

After you submit your task, Bob starts exploring.

You see the status change to "Todo List Created"

While Bob reads your code, it looks at:

- The GreetingResource class
- The GreetingResourceTest
- the pom.xml
- Existing validation (if any)
- Error handling patterns

Expect a short pause—on the order of seconds, not minutes.

Let that phase finish. Interrupting mid-read mostly buys you noise.

Bob is building a picture of how validation, errors, and tests already work in your project—not only the one endpoint.

What Bob is looking for

Roughly:

- How validation is currently done in your project
- What validation libraries you already use

- What error response format you follow
- What patterns exist for similar endpoints
- What tests exist for the greeting endpoint

Related files matter because patterns live next to the happy path.

Why this matters

Suggestions tend to mirror what already shipped: Bean Validation where you already use it, your error shape if you have one, tests that match neighboring endpoints.

Bob is not trying to win a style debate. It is trying to stay consistent with the codebase you showed it.

8.4. Reading Bob's Plan Before Execution

After exploration, Bob presents a plan as a todo list.

The status changes to "Waiting for approval."

Example shape:

> **Todo List Created**
>
> - *Add query parameter to accept name input in the greet() method*
> - *Add Jakarta Bean Validation dependency to pom.xml if not present*
> - *Add @NotBlank validation annotation to the name parameter*
> - *Add proper error handling for validation failures*
> - *Update tests to cover validation scenarios (valid name, null name, empty name)*
> - *Test the endpoint manually to verify validation works correctly*

That list is the contract for execution—what Bob intends to touch and in what order.

How to read the plan

Read it like a design note, not like prose to skim.

Ask:

- Does this approach make sense?
- Are these the right steps?
- Is this consistent with how we do things?

You still should not see raw implementation during planning—only what will change and why—so you can judge the approach before syntax pulls you sideways.

What to look for

Consistency: Does the plan follow your existing patterns?

Here Bean Validation shows up because the rest of the project already leans that way.

Completeness: Does the plan cover the failure modes you care about?

In this walkthrough, validation, error handling, and tests all appear—drop one and you can still build while production misbehaves.

Alternatives: Did Bob mention other approaches, or only one path?

If you do not understand the plan, or the risks are not yours to accept, stop and send Bob back to planning with sharper intent.

8.5. Understanding Bob's Guarantees

Before you approve, split what is mechanical from what still needs human judgment.

What Bob guarantees

On the mechanics side, Bob commits to:

- Following the approved plan
- Following patterns already visible in your repo

- Keeping edits syntactically correct
- Applying the batch atomically (all or nothing)
- Leaving a trace in task history

Those are controls on process. They are not proof that the change belongs in production.

What Bob doesn't guarantee

Bob cannot promise:

- That the change solves the real problem
- That side effects stay bounded in your environment
- That this was the best possible design
- That every edge case behaves
- That nothing regresses elsewhere

Those stay on you to test, review, and accept.

Your responsibility

Approval means you still own outcomes:

- Verify behavior in your environment
- Run the tests and manual checks that match the risk
- Review with the team when the blast radius is shared
- Ship and monitor on your terms

Bob sharpens deliberation. It does not move accountability off your desk.

The approval decision

You have three options:

Approve: The plan looks good. Execute it.

Modify: The plan is close but needs changes. Refine the intent and let Bob replan.

Reject: The plan doesn’t work. Start over with different intent.

Assume you approve.

Click "Approve and Execute."

8.6. Watching Execution

The status changes to "I’m working on it."

Bob starts making changes. You see progress:

> *Modifying: src/main/java/org/acme/GreetingResource.java Modifying: pom.xml Modifying: src/test/java/org/acme/GreetingResourceTest.java Running: tests to verify changes*

Execution should feel fast and dumb: files change, commands run, progress ticks.

If execution feels exciting, something went wrong—usually upstream of the merge.

If something goes wrong

Runs fail for boring reasons: someone else touched the same file, a test goes red, a merge conflict appears.

Bob stops, surfaces the error, and waits. It does not continue the run after a failure or auto-fix execution problems without you.

You can:

- Fix the issue manually and let Bob retry
- Modify the plan to work around the issue
- Cancel the task and start over

When mechanics break, you choose the recovery path.

When execution succeeds

When execution completes, Bob shows you:

> **Task complete: Add validation to the greeting endpoint**
>
> **Changes made:** *- Added @NotBlank on the name parameter in greet() - Added tests in GreetingResourceTest for valid, null, and empty input*
>
> **Verification:** *- All tests pass - Code compiles successfully - No linting errors*

That summary is what you verify against the real diff.

Your next steps

After Bob completes a task:

- Review the actual code changes
- Run your tests
- Test the functionality manually
- Check for unintended side effects
- Commit the changes (or don't)

Bob typed the edits; you still sign off on correctness.

8.7. What You Learned

You ran the loop: intent, exploration, plan, approval, execution, verification—the same rhythm The Agentic SDLC names. The slow beats are where judgment lives.

Try one small real task on your own before you read on. First passes are always clumsy.

Next, Context is the work keeps Bob's view accurate and bounded. After that, code review as evidence swaps roles: you already changed the code, and Bob brings findings so you decide what still needs fixing before ship.

Part III: Safe Change in Real Systems

Chapter 9. Code Review as Evidence

9.1. Finding Issues Without Losing Context

Earlier chapters lean on Bob to draft changes. Here the diff is already yours.

Review mode is evidence-first: Bob lists issues, explains risk, sometimes sketches a fix—you decide what ships and what waits.

Planning workflows still want an approved plan before edits. Review is different: you own triage and patch choices.

Below is how to start a review, read findings, sort by severity, and accept, defer, or reject without treating the panel as oracle.

9.2. Starting a Code Review

Start from a dirty workspace—your edits, a refactor, or a merge result you have not committed yet.

Run review before you commit when you want another pass on the diff.

Open the command palette and choose "Bob: Review Current Changes", or run `/review` in the Bob side panel.

Status moves to something like "Reviewing changes..." while Bob reads the delta.

It is building a picture of:

- What files moved
- What was added or modified
- How this repo usually handles validation, errors, and tests
- Whether security or quality issues show up in **this** change set

Expect seconds to tens of seconds—more when the diff is fat.

Let that pass finish. Interrupting mid-review mostly wastes the run.

9.3. Understanding Findings

When Bob finishes reviewing, findings appear in the Bob Findings panel.

Each finding has:

- A description of the issue
- The file and line number
- The severity level (high, medium, low)
- An explanation of the risk
- A suggested fix

Example: Security Finding

> ### SQL Injection Risk
>
> *File:* `UserService.java:47`
>
> *Severity: High*
>
> *The query concatenates user input directly into SQL. This allows attackers to inject malicious SQL commands.*
>
> ```
> String query = "SELECT * FROM users WHERE email = '" + email + "'";
> ```
>
> *Suggested fix: Use parameterized queries with PreparedStatement.*

That pattern is what you want from a finding: problem, consequence, plausible fix—grounded in lines that exist on disk.

What makes a useful finding

Strong findings read as:

- What failed (here: concatenated SQL)
- Why it bites (untrusted input becomes executable SQL)
- What to try next (parameter binding)

Useful findings tie back to your actual diff. Bob does not silently rewrite files in review mode and does not decide priority for you—it surfaces candidates; you accept or argue

with them.

9.4. How Bob Explains Risk

Risk is not a single bit. It blends likelihood, blast radius, exploitability, and fix cost—Bob's write-ups usually touch several of those axes.

Example: High Risk Finding

> ### Severity: High
>
> *This SQL injection vulnerability is easy to exploit. An attacker can steal user data, modify records, or delete the entire database. The fix is straightforward: use PreparedStatement instead of string concatenation.*

This is high risk because:

- Easy to exploit (any user can try it)
- Bad consequences (data theft, data loss)
- Simple fix (well-known pattern)

Treat that class of issue as blocking for commit unless you have a documented exception.

Example: Medium Risk Finding

> ### Severity: Medium
>
> *This method doesn't validate input length. Very long inputs could cause performance problems or memory issues. The fix requires adding validation and deciding on appropriate limits.*

This is medium risk because:

- Harder to exploit (requires specific conditions)
- Moderate consequences (performance degradation)

- More complex fix (requires business decisions)

Worth tracking; rarely worth stopping the world unless your threat model says otherwise.

Example: Low Risk Finding

> ### Severity: Low
>
> *This variable name doesn't follow your naming convention. It uses camelCase instead of snake_case. This inconsistency makes the code slightly harder to read.*

This is low risk because:

- Not exploitable (just a style issue)
- Minor consequences (slight readability impact)
- Easy fix (rename the variable)

Rename-or-ignore decisions belong with team taste and release pressure—not with Bob's severity tag alone.

Why risk levels matter

Severity is a sort key, not a substitute for judgment. Most teams fix high-severity security issues before merge, track medium items with owners and dates, and let low-severity noise wait until a cleanup window—unless policy says otherwise.

Bob sketches risk; your backlog and threat model set the actual bar.

9.5. Navigating Findings in the IDE

The Bob Findings panel is the index; the editor is where you decide.

Clicking a finding

Click a row and the editor jumps to the line. You usually get highlight, short explanation, suggested direction, and enough surrounding lines to see whether the warning applies **here**.

Why context matters

A finding without the surrounding code is just noise. With the diff in front of you, you can ask whether the risk is real in your deployment, whether the fix fits the architecture, and what else has to move if you change this spot.

Navigating between findings

Hop issue to issue from the panel or shortcuts—your muscle memory. Work high severity first when security is on the line, then medium, then low unless style rules say otherwise.

9.6. Security Findings and Quality Issues

Rough split: **security** findings are exploit-shaped or exposure-shaped; **quality** findings are maintainability-shaped.

Security findings

Examples teams see often:

- SQL injection vulnerabilities
- Cross-site scripting (XSS) risks
- Insecure authentication
- Exposed secrets or credentials
- Unsafe deserialization
- Missing input validation

Expect many of these to land as high or medium severity—default stance is fix before merge unless your process documents an accepted risk.

Quality findings

Examples:

- Inconsistent naming conventions
- Missing error handling

- Duplicate code
- Methods that should split
- Missing tests
- Unclear names

Often medium or low. Schedule them against debt budgets, not adrenaline.

Why the split matters

Security issues can hurt people and data in production. Quality issues mostly hurt the next developer—including future you. Order work accordingly.

9.7. When to Accept, Fix, or Reject Suggestions

Valid finding does not mean mandatory fix this minute—policy, schedule, and architecture still apply.

You have three common responses:

Accept and fix

The warning matches the code, the risk is real for your deployment, and the suggested direction fits.

Patch, test, commit—typical path for blocking-severity security items.

Accept but defer

The warning is directionally right, but the fix needs design time, another team, or a flag day.

Ticket it, note the risk, ship with eyes open only if your process allows that trade.

Reject

Bob misread intent, the pattern is deliberate, or compensating controls live elsewhere.

Dismiss with a short comment so the next reader knows why—reviews are social objects.

How to decide

Sanity-check accuracy, real-world risk for **your** threat model, fit of the proposed change, and whether now beats later. Evidence comes from Bob; priority stays with you.

9.8. Example: Reviewing a Real Change

Walk-through: new API endpoint, `/review` in the side panel, three synthetic findings to show the decision pattern—not a guarantee Bob always returns exactly three items.

Bob reports something like:

Finding 1: High Severity

> ### Missing Input Validation
>
> *The endpoint accepts user input without validation. This could allow malicious data to reach the database.*

You click the finding. You see your code:

```
@POST
@Path("/users")
public Response createUser(UserRequest request) {
    userService.create(request);
    return Response.ok().build();
}
```

The warning matches the snippet: no validation on the path into persistence.

You choose accept and fix.

You add validation:

```
@POST
@Path("/users")
public Response createUser(@Valid UserRequest request) {
    userService.create(request);
    return Response.ok().build();
}
```

Finding 2: Medium Severity

> ### Missing Error Handling

> *The endpoint doesn't handle exceptions. If the service throws an error, the client gets a generic 500 response instead of a meaningful error message.*

You click the finding. You see the same code.

The gap is real; fixing it properly means picking error shape and mapping—bigger than this diff.

You choose accept but defer.

You create a ticket: "Add proper error handling to user creation endpoint."

You add a comment in the code:

```
// TODO: Add error handling (see ticket #123)
```

Finding 3: Low Severity

> ### Method Name Convention
>
> *The method name `createUser` doesn't match your convention of using `add` for creation methods.*

You click the finding. You look at other endpoints.

Your conventions use `create` on public POST routes and `add` on internal helpers—intentional, not oversight.

You reject the finding.

You dismiss the finding. You add a comment:

```
// Note: We use 'create' for POST endpoints, 'add' for internal methods
```

What you learned

One fix, one defer, one reject—that distribution is normal. The task is not maximizing green checks; it is deciding what matters for this merge.

9.9. What You Learned

Same three ingredients as elsewhere—intent, evidence, judgment—except you supplied the diff and Bob supplied annotated findings. The skill is not zero warnings; it is knowing which warnings are real for **this** change.

Skills, modes, and rules as team memory covers slash commands, modes, and skills—habits that keep daily work fast without skipping intent. Modernization without behavior drift applies the same careful reading when you change code you did not write. Working with Bob as a team picks up visibility: how findings and plans read in PRs so nobody has to guess how the code got there.

Run `/review` on a branch you own and on one you barely know—the signal you need shifts; that is the point.

Chapter 10. Modernization Without Behavior Drift

10.1. Changing Code Without Breaking Systems

You already practiced day-to-day commands and skills in Skills, modes, and rules as team memory. After code review as evidence, you know how Bob annotates changes you already made.

This chapter is about changing code you did not write—or cannot fully load into your head—without quietly rewriting behavior.

Greenfield work has a spec in your head. Legacy work has behavior in production; your job is to keep that contract while the syntax catches up.

Bob is not a magic migrator. It is scaffolding for intent: map what exists, flag blast radius, split plans so syntax moves before semantics, and leave traces you can verify.

You will see how to read legacy behavior, coordinate multi-file work, and review diffs for drift—not how to “modernize” by trusting vibes.

10.2. The Modernization Problem

Legacy code is code that pays rent: old, messy, dated patterns, still correct enough to ship.

You touch it for features, security, dependency lifts, or performance—and the constraint is the same: do not trade correctness for pretty.

Why modernization is hard

You never get full visibility:

- Undocumented behavior and tribal knowledge
- Implicit dependencies and accidental coupling
- Edge cases that only show up under load or bad data
- Assumptions living in heads instead of tests

Change the program without that map and you are editing a system you only partly understand. That is the risk.

Why naive AI makes it worse

Full-file rewrites look tempting. Patterns match easily; contracts do not.

A generic “modernize this” pass often swaps idioms and accidentally swaps semantics. You only notice when production argues back.

Bob’s approach

Bob does not replace your judgment. It helps you stack evidence before diffs land:

- What the code does today
- What might break when it moves
- Plans that separate safe syntax from behavior changes
- Checks that behavior stayed put

Speed is not the pitch here; controlled change is.

10.3. Understanding Legacy Behavior First

Change nothing until you can say what the code **does**, not what the README wishes it did.

Starting with evidence

Picture a service you need to touch—Java 8 to 21, deprecated APIs, test seams, whatever the driver is.

Open with analysis, not edits:

"Analyze the UserService class and explain what it does."

Bob responds with a plan:

> *I’ll analyze the UserService class to understand its behavior. Let me examine:*
>
> - *The class structure and dependencies*
> - *All public methods and their behavior*
> - *Database interactions and queries*

> - *Error handling patterns*
> - *Side effects and state changes*
> - *Integration points with other services*

That opener is evidence gathering—read first, patch later.

What Bob finds

After analysis, Bob shows you:

> ### UserService Analysis
>
> **Core Behavior:** *- Creates users with email validation - Sends welcome emails after creation - Updates user profiles with audit logging - Deletes users with cascade to related records*
>
> **Dependencies:** *- UserRepository (database access) - EmailService (external service) - AuditLogger (logging service) - ValidationService (input validation)*
>
> **Important Patterns:** *- All database operations use transactions - Email failures don't block user creation - Audit logs are written synchronously - Validation happens before database access*
>
> **Potential Issues:** *- No retry logic for email service - Synchronous audit logging could slow operations - No pagination for user list queries - Exception handling is inconsistent*

Treat that output as ground truth to argue with—not scripture, but a structured read of what is on disk.

Why this matters

Before edits, lock what must stay true:

- Required behavior (create user, send mail paths)

- Odd but load-bearing quirks (email failure non-blocking)
- Performance smells that might still be contract (synchronous audit)
- Gaps you choose to fix later (retries, pagination)

Bob inventories; you pick preservation versus improvement.

10.4. Preserving Behavior While Refactoring

Once the inventory exists, intent can be specific.

Defining intent for modernization

Tell Bob something preservation-shaped:

"Refactor UserService to use modern Java patterns while preserving all existing behavior."

Bob asks for clarification:

> *I need to understand your priorities:*
>
> - *Should I preserve the synchronous audit logging behavior, or is it okay to make it asynchronous?*
> - *Should I preserve the "email failures don't block" behavior, or should we add retry logic?*
> - *Should I add pagination to list queries, or keep the current behavior?*
> - *What Java version are we targeting?*

Those questions are about behavior boundaries, not line noise.

The preservation plan

You clarify your intent:

"Preserve all existing behavior exactly. Use Java 21 features where they don't change behavior. Add pagination as a new optional parameter."

Bob creates a plan:

> **Modernization Plan**
>
> **Phase 1: Safe Refactoring (No Behavior Change)** - *Replace anonymous classes with lambdas - Use var for local variables where type is obvious - Replace null checks with Optional where appropriate - Use switch expressions for cleaner code - Add @Override annotations where missing*
>
> **Phase 2: Add New Features (Behavior Addition)** - *Add optional pagination parameter to list methods - Keep existing behavior when pagination not specified - Add tests for both paginated and non-paginated cases*
>
> **Preserved Behaviors:** - *Synchronous audit logging (unchanged) - Email failures don't block user creation (unchanged) - Transaction boundaries (unchanged) - Validation order (unchanged) - Exception handling patterns (unchanged)*
>
> **Verification:** - *All existing tests must pass without modification - Add new tests for pagination feature - Manual testing of email failure scenarios*

Phases matter because they separate **syntax-only** moves from **behavior additions**.

Phase 1 should survive old tests unchanged. Phase 2 needs new tests and sharper review. Approve Phase 1 when the contract is "shape only"; slow down when the contract moves.

10.5. Coordinating Multi-File Changes

Legacy changes rarely stay inside one buffer—dependencies fan out.

The dependency problem

Modernizing `UserService` drags along whatever it calls and whoever calls it:

- UserRepository (database access)
- EmailService (external service)
- AuditLogger (logging)
- ValidationService (validation)

And other services depend on UserService:

- UserController (REST API)
- UserEventHandler (event processing)
- AdminService (admin operations)

Touching the core service can force moves at every arrow.

Bob's multi-file analysis

Ask:

"What files will be affected if I modernize UserService?"

Bob analyzes the dependencies:

> ### Impact Analysis
>
> **Files That Must Change:** *- UserService.java (primary target) - UserServiceTest.java (tests need updates for new patterns)*
>
> **Files That Might Need Changes:** *- UserController.java (if we change method signatures) - UserEventHandler.java (if we change event handling)*
>
> **Files That Should Not Change:** *- UserRepository.java (interface stays the same) - EmailService.java (interface stays the same) - AuditLogger.java (interface stays the same)*
>
> **Recommendation:** *Keep UserService interface unchanged. Refactor internal implementation only. This limits impact to*

> *UserService and its tests.*

That is blast-radius mapping—what must move, what might move, what should stay put.

The coordination plan

Based on this analysis, you refine your intent:

"Refactor UserService internal implementation only. Keep all public methods unchanged."

Bob creates a coordinated plan:

> **Multi-File Coordination Plan**
>
> **Step 1: Refactor UserService internals** - *Change private methods to use modern patterns - Keep public method signatures unchanged - Keep return types unchanged - Keep exception types unchanged*
>
> **Step 2: Update UserServiceTest** - *Add tests for new internal patterns - Keep existing test cases unchanged - Verify all existing tests still pass*
>
> **Step 3: Verify dependent files** - *Run tests for UserController - Run tests for UserEventHandler - Confirm no changes needed*
>
> **Safety Checks:** - *No changes to public API - No changes to database schema - No changes to external service calls - No changes to error handling behavior*

Stable public surfaces shrink the edit cone—internals can move while controllers and neighbors stay boring.

Bob's job here is to surface the **smallest** edit that meets the intent, not the largest refactor that fits in one afternoon.

10.6. Reviewing Modernization Changes Safely

After Bob lands edits, the review is about **drift**, not just style.

The verification checklist

For modernization, walk this in order:

Syntax Changes Only

Look at the diff. Are the changes purely syntactic?

- Anonymous classes → lambdas (safe)
- Explicit types → var (safe)
- if-else chains → switch expressions (safe)
- null checks → Optional (verify carefully)

Pure syntax churn usually survives unchanged tests; mixed semantics might still slip through—stay skeptical.

Behavior Preservation

Run existing tests without edits when possible. Changing tests to match the diff is a signal: either the contract moved or the tests were wrong—know which.

Edge Cases

Pressure-test mental models: null and empty inputs, failing downstreams, load-ish ordering assumptions.

Integration Points

Compare queries, outbound calls, error shapes, and logs before versus after. Silent shifts there are often the real story.

Example: Reviewing a modernization

Bob refactored `UserService`; you read the diff line by line.

You see this change:

```
// Before
```

```
public User createUser(String email, String name) {
    if (email == null || email.isEmpty()) {
        throw new IllegalArgumentException("Email required");
    }
    if (name == null || name.isEmpty()) {
        throw new IllegalArgumentException("Name required");
    }
    // ... rest of method
}

// After
public User createUser(String email, String name) {
    if (email == null || email.isBlank()) {
        throw new IllegalArgumentException("Email required");
    }
    if (name == null || name.isBlank()) {
        throw new IllegalArgumentException("Name required");
    }
    // ... rest of method
}
```

Looks like a trivial modernization (`isEmpty()` → `isBlank()`).

It is not—the latter trims semantic whitespace; validation boundaries move.

Decide explicitly: bugfix, tightening, or mistake—then align tests and docs.

When to accept behavior changes

Improvements happen—just make them visible: note them in the PR, adjust tests on purpose, and keep risk owned by humans.

10.7. Example: Modernizing a Legacy Service

Worked example: `PaymentService` on Java 8, ugly but stable—lift syntax without shaking guarantees.

Step 1: Understand current behavior

You ask Bob:

"Analyze PaymentService and document its behavior."

Bob examines the code and responds:

> ### PaymentService Behavior

> **Core Operations:** - *Processes credit card payments - Validates card numbers using Luhn algorithm - Stores payment records in database - Sends confirmation emails - Handles payment failures with retry logic*
>
> **Critical Behaviors:** - *Retries failed payments 3 times with exponential backoff - Rolls back database changes if email fails - Logs all payment attempts for audit - Returns specific error codes for different failure types*
>
> **Dependencies:** - *PaymentGateway (external service) - PaymentRepository (database) - EmailService (notifications) - AuditLogger (compliance)*

That narrative is the preservation contract.

Step 2: Define modernization intent

You tell Bob:

"Modernize PaymentService to Java 21. Preserve all existing behavior. Use modern patterns where safe."

Bob asks:

> *Should I: - Keep the 3-retry logic exactly as is? - Keep the rollback-on-email-failure behavior? - Keep the specific error codes? - Add any new features?*

You respond:

"Keep all behavior exactly as is. No new features. Just modernize the code."

Step 3: Review the plan

Bob creates a plan:

> ## Modernization Plan

> **Safe Changes:** *- Replace anonymous classes with lambdas - Use var for obvious types - Use switch expressions for error code mapping - Use text blocks for SQL queries - Add @Override annotations*
>
> **Preserved Behaviors:** *- 3-retry logic (unchanged) - Exponential backoff timing (unchanged) - Rollback-on-email-failure (unchanged) - Error code mapping (unchanged) - Audit logging (unchanged)*
>
> **Verification:** *- All 47 existing tests must pass unchanged - Manual test of retry logic - Manual test of rollback behavior - Review audit logs for format changes*

You review this plan. You approve it.

Step 4: Execute and verify

Bob makes the changes. You review the diff.

You see:

- Lambdas instead of anonymous classes (safe)
- var instead of explicit types (safe)
- Switch expressions instead of if-else (verify carefully)
- Text blocks for SQL (verify carefully)

You run the tests. All 47 tests pass without modification.

You test the retry logic manually. It works exactly as before.

You test the rollback behavior. It works exactly as before.

You check the audit logs. The format is unchanged.

If checks line up—tests green where they should be, tricky paths exercised—the modernization story closes.

What you learned

Same recipe: inventory, preservation intent, phased plan, diff discipline, verification.

Bob supplied reads and diffs; humans owned go/no-go.

10.8. When Modernization Goes Wrong

Green tests do not guarantee calm production. Timing, exceptions, and dependency defaults can move while suites stay polite.

Common modernization failures

Timing changes — lambdas, streams, or ordering tweaks can reorder side effects without failing unit coverage.

Exception handling changes — new wrappers change what bubbles and what gets swallowed.

Dependency changes — upgraded APIs and defaults shift behavior in paths tests never touch.

How Bob helps catch these

Bob will not certify safety end to end. It can still stack artifacts: the diff, impact hints, and the checklist you agreed on—so humans hunt drift instead of staring at green alone.

Your responsibility

Plans and merges stay yours. Treat Bob like tooling that amplifies visibility, not a rubber stamp.

10.9. What You Learned

Modernization here means **preservation first**. Inventory legacy behavior, spell what must stay true, keep edits small and phased, read diffs for semantic drift, and accept intentional behavior shifts only with explicit notes and tests.

Risk does not disappear—Bob shrinks surprise by forcing evidence before and after edits.

Security, permissions, and MCP trust is next—audit trails matter most when every change needs a story.

Try one legacy surface you actually fear: analyze first, change second. The skill is keeping behavior stable while the code catches up—not chasing “modern” for its own sake.

Chapter 11. Security, Permissions, and MCP Trust

11.1. Shift Left Without Pretending Risk Disappeared

Security belongs in how you work every day—not only as a gate before release. Bob can help you catch mechanical issues early, document intent, and keep plans reviewable, but **agents read your repo, run shell, call APIs, and use MCP**. Treat that stack like privileged software, not a fancy autocomplete.

This chapter aligns with IBM Bob security guidance: what goes wrong, how to contain damage, which product controls exist, and how auditability fits regulated work.

11.2. What Actually Goes Wrong

Unlike classic apps with mostly fixed code paths, agents **interpret text and choose tools**. Defenses have to cover **behavior**, not only network edges.

- **Too much power** — If the agent can run shell, reach production controls, or read secrets for a task that only needed a summary, mistakes or manipulation scale fast.
- **Hidden instructions in data** — Malicious or careless text in repos, issues, skills, or mail can steer tool use. Treat ingested content as untrusted.
- **Internal access via the agent** — Fetches and integrations can reach metadata services or internal APIs the user never meant to expose.
- **Supply chain** — A compromised connector (MCP package, plugin update) runs with whatever trust you gave the agent.
- **Over-trusting the model** — Confident wrong answers still need human review for high-stakes decisions.

Design so that **even bad prompts or poisoned context cannot do unlimited damage**—containment beats hoping the model always behaves.

11.3. Approvals, YOLO Mode, and “Safe” Commands

YOLO mode (everything auto-approved) means shell, files, and network can run **without a human pause**. It is fast until it is not—one injected prompt can act at machine speed.

Hybrid setups sometimes auto-approve “low-risk” steps. That breaks down for the

terminal: the risk is rarely the binary name alone—it is the **arguments**. Allowlisting a command name does not stop hostile flags on the same binary.

Do this

- Turn off blanket auto-approval for shell, outbound network, and infrastructure-changing tools unless you have proven controls.
- Prefer granular prompts per action.
- Do not rely on executable-only allowlists until you validate full command lines—or keep those paths manual.

This matches how you already treat Context is the work: trim what the agent sees, and treat auto-run settings as part of the threat model.

11.4. The Permission Ladder

Think about agent permissions as a ladder, not a binary switch.

1. Read files and explain
2. Summarize and plan
3. Edit files
4. Run tests
5. Run shell commands
6. Use network
7. Use MCP tools
8. Touch shared environments
9. Prepare production changes
10. Execute production changes

The rule is simple: climb only when the task justifies it, and never several rungs at once just because the model sounds confident.

11.5. Shared Vocabulary: OWASP Top 10 for LLM Applications (2025)

When you talk to security, audit, or governance, use the standard names and numbering from the OWASP GenAI project (**LLM01** through **LLM10**). Cite the edition you mean—older PDFs used different numbering.

- **LLM01** Prompt injection
- **LLM02** Sensitive information disclosure
- **LLM03** Supply chain
- **LLM04** Data and model poisoning
- **LLM05** Improper output handling
- **LLM06** Excessive agency
- **LLM07** System prompt leakage
- **LLM08** Vector and embedding weaknesses
- **LLM09** Misinformation
- **LLM10** Unbounded consumption

In the 2025 list, **LLM06** is **Excessive agency**—granting the model more ability to act than the task requires. Your mode choice and MCP configuration are levers against that.

11.6. Layered Containment: Where Code Runs, Network, Secrets

Assume the agent will eventually see malicious or misleading context. **Goal:** limit how far a bad turn can spread.

- **Workspace discipline** — Work inside a defined project workspace, not arbitrary user folders.
- **Sandboxes** — Devcontainers or dedicated sandboxes for agent-heavy work beat a single overloaded laptop session with full user rights.
- **Network** — Default-deny outbound traffic where policy allows; allow only hosts the task needs.

- **Credentials** — Avoid long-lived API keys in files the agent routinely reads; prefer vaults, managed identity, and short-lived tokens.

Bob-specific layers—modes, `.bobignore`, checkpoints—complement these; they do not replace them.

11.7. Modes, Ignore Files, and Checkpoints (Bob)

- **Modes** cap which tool groups exist for this chat. For unfamiliar or untrusted code, start in **Ask** or **Plan**; move to **Code** when you intend to edit. Reserve **Advanced** (and skills) for people and tasks that truly need the full surface. **Orchestrator** coordinates larger efforts; it is not “all tools in one blind bundle.”
- **`.bobignore`** — Keep secrets, keys, and noisy paths out of routine context, alongside the context rules in Context is the work.
- **Checkpoints** — Snapshot before writes land; restore files or roll back a bad turn without a manual Git rescue (still not a substitute for commits).

11.8. MCP and Enterprise Integration

MCP connects Bob to external systems. Configuration is part of your attack surface.

- Configs live in user and project MCP settings (typical files: user `~/.bob/mcp_settings.json`, project `.bob/mcp.json`—confirm paths in IBM Bob documentation). Treat them like **production integration configs**—review in pull requests, and do not commit long-lived secrets as plain strings.
- The **alwaysAllow** (or equivalent) list runs named tools **without** an approval prompt—a small YOLO. Keep it **empty** for anything that writes, runs commands, or touches sensitive systems until policy explicitly allows otherwise.
- **Gateway and vault patterns** — Put MCP behind policy; mint **short-lived** credentials per call; revoke centrally if something looks wrong.
- **On-behalf-of (OBO)** — Downstream systems should see the **developer’s identity**, not an overpowered shared service account.

Threat-model each new MCP server and mode. Version system and custom prompts like code. If you use retrieval or embeddings, review data and retrieval separately from “plain” chat. Monitor tool calls and egress with alert budgets your team can act on.

Each MCP server is a bridge into another system—credentials, scopes, and silent tool runs deserve the same skepticism as shipping a new microservice dependency. When you want a tighter tour of MCP-specific hazards and review habits than this chapter covers, continue from IBM Bob security guidance.

11.9. Security habits for product and non-engineering roles

You do not need to patch servers to influence risk: prompts, pasted customer text, screenshots of roadmaps, and whole attachments shape what the agent tries to do next. Product managers, designers, writers, and customer-facing engineers still benefit from verification-first habits—small bounded asks, lighter attachments, and clarity about what must never leave controlled environments.

If your squad mixes engineers with PMs, designers, or writers—not everyone maintains YAML—use IBM Bob security guidance next to Context is the work when the risky part is what entered the chat, not only which tools ran.

11.10. Rollout Order (Practical Checklist)

A sensible default sequence for teams:

1. **Pick conservative modes by default** — Plan for design, Ask for exploration; tightly control who uses Advanced.
2. **Require human approval** for shell, network, and infra unless you have proven argument-level controls—not binary allowlists alone.
3. **Ship a team `.bobignore`** that covers environment files, keys, credentials, and internal-only paths.
4. **Harden MCP** — gateway, vault-issued secrets, OBO; leave risky tools off silent auto-run lists.
5. **Scan generated code** in the IDE before merge—catch secrets and unsafe patterns early.

11.11. Compliance and Auditability with Bob

Regulated environments need traceability. Bob's workflow helps when you use it deliberately:

- **Intent and plans** — Document what you asked for and what Bob proposed before

execution; paste plans into tickets or pull requests (see Working with Bob as a team).

- **Task history** — Searchable history supports “who asked what, what was approved, what changed.”
- **Human judgment on the record** — Approvals, deferrals, and rejections of findings are decisions **you** own; the paper trail should show that.

Bob does not replace your compliance program. It supplies **evidence packets**—plans, findings, diffs—so you can explain changes to auditors and reviewers. You still define retention, classification, and what may not enter third-party systems at all per your org’s rules.

11.12. What You Learned

Agents need the same seriousness as production software: least privilege, containment, careful MCP configuration, and no silent auto-run on powerful tools. Standard OWASP LLM labels give you a shared language with security. For Bob, modes, `.bobignore`, and checkpoints are day-to-day guardrails—paired with workspace discipline and secrets hygiene.

Go deeper in IBM Bob documentation:

- Security guidance
- Context mentions

Working with Bob as a team covers how those habits scale when everyone shares the repo.

Chapter 12. What Bob Should Not Do

12.1. Where Human Judgment Is Mandatory

Earlier chapters showed where Bob fits. This one draws the line: decisions, final review, context, deploy buttons, and constraints you state out loud still belong to people.

The product encodes non-goals—you still fill in what “safe” means for your system.

12.2. Bob Should Not Make Decisions for You

First boundary: Bob does not own the call—evidence and options, yes; accountability, no.

You still own:

- What code to write
- What risks to accept
- What trade-offs to make
- What ships to production
- What matters to your users

Judgment needs local context and someone’s name on the line—Bob supplies neither.

Suggestions still need your read: possibility is cheap; “right for us” is not automatic.

Example: The automatic approval trap

You ask for performance; Bob proposes in-memory caching of user data.

The idea can be sound on paper—faster reads—without knowing your memory ceiling, consistency rules, eviction story, or privacy bar.

Map those constraints, pick the trade-off, own the outage if the cache lies.

How to avoid this trap

Before you click approve on a plan:

- Can you explain the suggestion in one plain sentence?

- Why this approach instead of the next obvious option?
- What breaks if load, data, or operators behave badly?
- Would you defend this in standup without hiding behind the tool?

If any answer is fuzzy, pause—ask for alternatives, shrink scope, or think longer without Bob in the loop.

12.3. Bob Should Not Replace Code Review

Bob can surface mechanical risk early; it does not retire human review.

What Bob tends to catch

- Security vulnerabilities
- Code quality issues
- Pattern violations
- Missing tests
- Inconsistent naming

Pattern and static-shape problems sit in Bob's wheelhouse.

What still needs humans

- Whether the business logic matches reality
- UX and workflow friction
- Behavior under real load and messy integrations
- Tacit team knowledge ("we never call this on Fridays")

Bob is a fast lint-plus for the diff; humans still answer "should we ship this experience?"

Example: The false sense of security

Imagine Bob clears your auth change for obvious injection-style issues. You skip peer review because the panel looked quiet.

The login might still be technically "secure" and practically hostile—reset flows, lockouts, copy that confuses people. That failure mode is not a CVE line item.

How to use Bob in code review

Treat Bob as pre-review:

1. Make changes
2. Run Bob's review and fix what you agree with
3. Hand the same diff to people for product and operational judgment
4. Fold their feedback in
5. Commit when both passes mean something

Machines flag patterns; humans sign off on outcomes.

12.4. Bob Should Not Write Code Without Context

Bob can emit code; without your repo's grammar it emits generic Java-shaped paste.

The context problem

A naked "write a user service" omits architecture, naming, errors, tests, and taste that took years to sediment.

Code that compiles but fights house style is debt with a green build.

How to provide context

Before you ask for new surface area:

1. Point at existing code that already does the job well enough
2. Say which patterns are mandatory versus optional
3. Name hard constraints (transactions, logging, error contracts)

Use mentions such as:

- `@/src/services/UserService.java` (existing service shape)
- `@/src/test/` (testing habits)
- `@/.bob/rules/` (documented standards)

Context first; generation second.

Example: The mismatched code

You ask Bob: "Create a payment service."

Bob generates:

```
public class PaymentService {
    private PaymentGateway gateway;

    public PaymentService(PaymentGateway gateway) {
        this.gateway = gateway;
    }

    public PaymentResult processPayment(Payment payment) {
        return gateway.process(payment);
    }
}
```

This snippet compiles; it still ignores how your stack actually wires beans, wraps transactions, logs money paths, and surfaces failures.

Give Bob an exemplar:

"Look at `@/src/services/UserService.java` and create a payment service following the same patterns."

Then the diff inherits injection style, annotations, transaction boundaries, exceptions, and audit hooks instead of inventing a parallel universe.

12.5. Bob Should Not Be Your Unsupervised Production Operator

Bob can build, test, containerize, shell out, and drive MCP—which is why it must not become an autopilot deploy button.

Production touches calendars, neighbors' releases, paging policies, and blast radius Bob cannot see from the repo alone.

What Bob can do

Strong fit for prep work:

- Generate manifests and config drafts

- Script repeatable steps
- Document runbooks
- Sanity-check readiness checks

Also fine for explaining logs or rehearsing rollback wording—still with a human holding the pager.

The apply button stays yours: execute, watch signals, own incidents.

Example: The automated deployment disaster

Picture auto-deploy on green tests while a dependency is yellow, another team is mid-flight, metrics are blind, and nobody is holding the incident keyboard. The pipe "succeeds"; customers still lose trust.

How to use Bob for production work

Let Bob draft and rehearse; humans schedule, approve, and watch:

1. Draft manifests or scripts
2. Review like any other code
3. Exercise in staging
4. Pick the window with operators in the loop
5. Execute production changes deliberately
6. Monitor and debrief

Preparation scales; accountability does not move to the tool.

12.6. Bob Should Not Ignore Your Constraints

Bob can propose clever work—only humans know which edges are hard walls.

What counts as constraints

Budget, time, headcount, stack choices, regulators, performance floors, security policy—non-negotiables you state up front.

If Bob optimizes in a vacuum, you get fantasy architecture.

Why spelling them matters

Unbounded prompts invite unbounded answers: new language, new fleet, new calendar—each maybe valid somewhere else, useless here.

Lead with the cage:

"Improve user-service latency. Constraints: no new boxes, no schema migrations, ship inside one sprint."

Example: The impossible suggestion

Ask for “10× traffic” without limits and you may get a conference talk—rewrite, cache farm, event sourcing—while you needed indexing and query fixes inside the stack you already run.

Reload the question with real bounds; answers shrink to things you can schedule.

How to fix this

Repeat constraints in the task until they feel boring—that is when Bob stops auditioning for a greenfield rewrite.

12.7. Anti-Patterns When Using Bob

Seven ways teams turn acceleration into liability:

Anti-Pattern 1: The Copy-Paste Developer

Ship Bob’s diff without reading it and you inherit bugs you cannot explain.

How to avoid: Read, run, and be ready to justify every line you keep.

Anti-Pattern 2: The Approval Bot

Rubber-stamp plans because they look confident and you want the inbox clear—you outsourced judgment while keeping the pager.

How to avoid: Question scope, alternatives, and failure modes before every approve.

Anti-Pattern 3: The Context-Free Requester

Ping-pong “fix it” without `@` mentions, rules, or exemplars—each round invents a new wrong shape.

How to avoid: Ground the task in files and constraints before you ask for edits.

Anti-Pattern 4: The Blame Shifter

When prod misbehaves, blame the model. The merge still has your name on it.

How to avoid: Own outcomes from tools you authorized.

Anti-Pattern 5: The Black Box User

Use Bob off-thread and leave teammates guessing how the code appeared.

How to avoid: Put intent, plans, and tool use where reviewers already look—commits, PRs, tickets.

Anti-Pattern 6: The Shortcut Seeker

Ask for answers to skip building mental models—you stay dependent on whatever the panel prints next.

How to avoid: Use explanations as teaching aids; verify claims against the repo.

Anti-Pattern 7: The Production Experimenter

Point automation at prod without staging, review, or an operator—speed without containment.

How to avoid: Same guardrails as any privileged change: review, staging when possible, humans on approvals, tight MCP and auto-approve posture around sensitive paths.

12.8. When to Stop Using Bob

Bob is not wrong for every situation—it is the wrong **first** move for some.

Be careful using Bob when:

You’re learning something new

Typing your own ugly code first teaches constraints the model will happily hide. Let Bob tutor after you have felt the edges.

You’re prototyping

Exploration benefits from cheap thrash; don’t let the first polished answer lock the

design. Bob can help brainstorm—still discard freely.

You're in an emergency

Latency and judgment beat chat latency. Use Bob for log triage if it helps; do not let it run incident command.

You don't understand the problem

Implementation without a bounded problem ships guesses. Analyze until the failure mode is named; then automate fixes.

You're making architectural decisions

Bob can spell options and trade-offs; ownership stays with people who will maintain the consequences.

12.9. Stop Conditions

Agentic work can spiral long before it obviously fails.

Stop when:

- Bob repeats itself without adding evidence.
- The plan changes without new information.
- You cannot explain the diff in plain language.
- The context window is crowded with stale or conflicting material.
- You are asking Bob to compensate for unclear intent.
- The tool asks for broader permissions than the task deserves.
- You are tired enough that approval has become mechanical.

12.10. What You Learned

Ownership stays human: decisions, final review, context, deploy authority, and explicit constraints. The anti-patterns are ordinary engineering failures—paste-only commits, rubber approvals, hidden AI use, blame shifting—accelerated when automation removes friction.

The Bob mindset carries the same discipline into everyday prompts. Know three

decisions you refuse to outsource; keep them in mind when the approve button glows.

Reach for Bob often; still lead with humans when you are learning cold, fighting fires, or placing long bets on architecture.

Part IV: Teams, Roles, and Governance

Chapter 13. Working with Bob as a Team

13.1. From Individual Tool to Shared System

Solo use is simple: you own the intent and the click on Approve. A team adds one requirement—everyone can see **how** Bob was used so PRs are not mystery boxes.

Bob is shared infrastructure, not a private autocomplete. Align on modes, rules, and what you paste into descriptions; mismatched habits show up as style churn long before anyone argues about the AI.

The sections below cover visibility in reviews, onboarding, shared steering (modes, `AGENTS.md`, rules), and patterns that keep usage boringly consistent—Modes, Skills, and Slash commands in IBM Bob documentation cover how the product encodes steering layers such as skills and team commands.

13.2. The Team Problem with AI Tools

Solo AI use hides process; team review needs traceability.

The black box problem

Reviewers see the diff and the tests—often not the prompts, retries, or discarded plans behind them. Reasonable-looking merges can still be unexplained work.

The consistency problem

Different people steer the model with different adjectives—fast, safe, simple—and get different patterns. The repo drifts without a meeting about it.

The knowledge problem

The person who pair-programmed with the model holds the story. Everyone else reverse-engineers intent from bytes. Review and maintenance slow down.

Bob’s approach

Bob is built around visible steps: stated intent, evidence the tool read, the plan you approved, the edits that ran, and risks called out. That is the shared record—less mystery in the merge, less archaeology later.

13.3. Shared Understanding, Not Hidden AI Work

Default: show the work—intent, plan, and what Bob did—where reviewers already look.

Document the intent

When you use Bob to make changes, document the intent in the commit message or pull request description.

Don’t write:

> *"Refactored UserService"*

Write:

> *"Refactored UserService to use modern Java patterns while preserving all existing behavior. Used Bob to analyze dependencies and coordinate changes across UserService and UserServiceTest."*

Reviewers get goal, involvement, and division of labor in one scan.

Share the plan

When Bob creates a plan, include it in the pull request description.

Copy Bob’s plan into the PR:

> ### Bob’s Plan:
>
> *Phase 1: Safe Refactoring (No Behavior Change) - Replace anonymous classes with lambdas - Use var for local variables where type is obvious - Use switch expressions for cleaner code*
>
> *Phase 2: Verification - All existing tests must pass without modification - Manual testing of edge cases*

They can compare the plan to the diff instead of guessing.

Share the findings

When Bob surfaces issues during review, surface them in the thread—not only in your local fixes.

Example shape:

> *Bob identified a potential SQL injection risk in line 47. Changed to use PreparedStatement. See Bob's finding:*

Other reviewers learn what almost shipped.

Why visibility matters

Shared traces beat tribal knowledge: people can judge outcomes and improve the habit—not fight over whether AI was involved at all.

13.4. Reviewing Bob's Output in Pull Requests

Bob can help open a pull request and polish the narrative. When the title and description look tidy, it is easy to skim past the hard part: **does the story match the diff?** Read the code; read the PR body as a separate claim about that code.

See pull requests and slash commands in IBM Bob documentation for current `/create-pr` and `/review` behavior.

Built-in PR creation (/create-pr)

You need commits on a branch and push access to the remote. Start from the PR icon in Source Control, **Bob: Generate PR** in the command palette, or `/create-pr` in chat.

Bob reads the branch diff, commits, and branch name, then proposes a title and description. You pick base branch and remote, edit if you want, create the PR, and get the link back in chat. The product surfaces a short list of likely base branches so you are not hunting the full inventory.

The description is built from what is on the branch—helpful, branch-local context. It is not a warranty; cross-check it against the diff.

`/review` before or beside the PR

Built-in `/review` runs a structured pass in chat (bugs, security, performance, style—see product documentation). Treat it as an extra pair of eyes on the diff, not a substitute for

reading the PR narrative.

- `/review` — local **uncommitted** changes (Code review as evidence).
- `/review <branch>` — compare that branch against your current branch (HEAD).
- `/review #<issue-number> --issue-coverage` — check local changes against a GitHub issue.
- `/review <issue-url> --issue-coverage` — same with an issue URL.

`/review` output is analysis from the context Bob could see—not proof the story matches the diff or that CI covered the risk. Walk intent → diff → verification when you approve.

Canonical syntax: Built-in slash commands in IBM Bob documentation.

PR templates

If the repo already ships a template, Bob follows it. Usual paths include `pull_request_template.md`, `docs/pull_request_template.md`, `.github/pull_request_template.md`, plus other locations under `docs`—more than one match and Bob asks which file to use.

If nothing matches, you get Bob's fallback shape: title tied to the branch name, summary, implementation notes, testing, links—edit in the temporary Markdown buffer and finish when ready.

Review in this order

A draft saves authors time; completeness and honesty are still on them. For a Bob-assisted PR, walk these in sequence:

1. **Stated intent** — What does the PR claim it is doing?
2. **Actual diff** — What actually changed?
3. **Verification story** — What ran, and does the text prove it?
4. **Missing risk** — What should worry us that never made the summary?

If the description is polished but vague, the prose got ahead of the engineering evidence.

Authors: before you ship

Bob can draft the shell; you still own how easy this is to review.

- Use a descriptive branch name—Bob derives part of the framing from it.
- Read the generated title and body before you submit. Generated is not the same as checked.
- Strip empty prompts, stale placeholders, and soft claims the diff does not support.
- Say what the change was **for**, not only which files moved.
- Carry important context from planning or review chats into the PR in plain English.
- For refactors: say what was supposed to stay unchanged.
- Make testing concrete—name tests, checks, or manual steps that actually ran.
- Add issues, screenshots, rollout notes, or follow-ups if the draft skipped them.

Bob saves typing; you keep judgment.

Reviewer checklist

Story versus diff. Read title and description, then the diff. Red flags: cleanup language next to behavior changes; "refactor only" with API or test churn; confident tone with thin proof; a Testing section that names nothing. When story and diff disagree, trust the diff.

Intent. Ask for clear answers before you approve: what problem this fixed, what behavior stayed put, what changed, what stayed out of scope.

Verification. A testing heading is cheap. Ask what actually ran, why tests moved, whether anyone exercised it by hand, and what "green" means in this repo.

Template shape. Sections can be full and still empty: Summary restates the branch name; Testing lists generic steps; Related issue is blank for ticket-driven work; Risks says nothing on a risky change.

Hidden behavior change. Push back when the narrative says modernization or cleanup but the surface area changed, tests moved without a stated reason, certainty runs ahead of evidence, or big files never show up in the text.

Questions worth pasting into review:

- Which behavior had to stay the same?
- What convinced you this was safe?
- Why did these tests change?
- What should I check beyond CI green?
- What would you flag if this merged tonight?

A practical review flow

1. Optional: run `/review` (or branch or issue variants) for a chat-side pass—then still read the PR.
2. Read the generated title and description.
3. See whether the template sections say something concrete.
4. Scan the diff for behavior risk.
5. Compare the testing story to what you believe ran.
6. Ask wherever the wording outruns the proof.
7. Approve when the body, diff, and verification line up.

Bob assists PR **creation**. Accuracy, risk, and merge calls stay human. Read generated PR text the same way you would a patch from a sharp teammate who was rushing.

Official docs: pull requests · slash commands (`/review`, `/create-pr`).

What still applies from any code review

Standard checks still apply: does it work, is it tested, is it maintainable, does it follow team conventions? Layer the Bob-specific checks above—intent versus diff, plan versus outcome, evidence, behavior preservation, documented risk—when the change involved Bob.

Example: Reviewing a Bob-assisted PR

You receive a pull request:

> **Title:** *Modernize PaymentService to Java 21*

> **Description:** *Used Bob to modernize PaymentService while preserving all existing behavior.*
>
> **Bob's Plan:** *- Replace anonymous classes with lambdas - Use var for obvious types - Use switch expressions for error handling - Keep all public APIs unchanged - Keep all retry logic unchanged*
>
> **Verification:** *- All 47 existing tests pass without modification - Manual testing of retry scenarios completed - No changes to public API*

You review the code. You check:

- Intent is clear (modernize while preserving behavior) ✓
- Plan is appropriate (safe syntactic changes) ✓
- Evidence was considered (mentions retry logic, public API) ✓
- Behavior is preserved (all tests pass unchanged) ✓
- Risks are acknowledged (explicitly states what was kept unchanged) ✓

You look at the diff. You see lambdas, var, switch expressions. All syntactic changes.

You approve the PR with a comment:

> *"Looks good. Appreciate the clear documentation of what was preserved. The switch expression in error handling is much cleaner than the old if-else chain."*

Say clearly when the PR story matches the diff—positive feedback reinforces the habit.

When to push back

Ask for more detail when:

Intent is unclear

> *"Can you clarify what behavior you were trying to preserve? The PR*

> *description mentions 'all existing behavior' but the tests were modified."*

Plan seems wrong

> *"Why did Bob suggest changing the public API? That seems risky for a refactoring task."*

Evidence was missed

> *"Did Bob check the impact on AdminService? That service also calls these methods."*

Behavior changed unexpectedly

> *"The tests pass, but the error messages changed. Was that intentional?"*

Risks weren't considered

> *"What about the timing changes from using streams? Did Bob analyze the performance impact?"*

Hard questions in review are how teams align—not how they shame authors.

13.5. Using Bob for Onboarding and Knowledge Transfer

Onboarding is faster when newcomers can ask for orientation without pretending they memorized every package.

Understanding the codebase

Pair reading with targeted asks—for example:

"Analyze the UserService and explain what it does."

Typical answer shape:

- Overview of the service
- Key dependencies
- Important patterns
- Potential issues

Treat it as a map, not scripture—spot-check against the repo.

Learning team patterns

During real edits, Bob tends to mirror what already shipped; newcomers feel house style through repetition—still verify against reviewers.

Safe experimentation

Before risky edits, rehearse impact—for example:

"What would happen if I change this method signature?"

Bob analyzes the impact:

> **Impact Analysis:**
>
> *This method is called by: - UserController (3 places) - AdminService (1 place) - UserEventHandler (2 places)*
>
> *Changing the signature would require updating all 6 call sites.*

Cheap rehearsal beats fixing six call sites after the fact.

Onboarding checklist

When bringing someone up with Bob:

1. **Show them how to ask for analysis**
    - "Analyze this service"
    - "Explain this pattern"
    - "What does this code do?"

2. **Show them how to define intent**
    - Be specific about goals
    - Describe desired outcomes
    - Let Bob figure out implementation
3. **Show them how to review plans**
    - Check for consistency with team patterns
    - Verify evidence was examined
    - Confirm risks were considered
4. **Show them how to verify changes**
    - Run tests
    - Check for unintended impacts
    - Review with the team
5. **Show them how to document**
    - Include intent in commits
    - Share plans in PRs
    - Document decisions

Same habits across people beats hero users with secret prompts.

13.6. Avoiding "AI Silos" in Teams

Silos show up when half the squad ships Bob-assisted diffs and half never touches it—speed and style diverge, then blame follows.

Why friction appears

Comfort levels differ; so do use cases (refactor versus review versus analysis). Without written norms, those differences become competing dialects in one tree.

Preventing silos

Write the boring stuff down:

Establish team guidelines

Document how your team uses Bob:

- When to use Bob (refactoring, code review, analysis)
- When not to use Bob (quick fixes, prototypes)
- How to document Bob usage (commit messages, PR descriptions)
- How to review Bob output (what to check, what to verify)

Ship it where onboarding and review already look: wiki, `CONTRIBUTING`, PR template callouts.

Share examples

Keep a few reference PRs or snippets: strong intent, strong plan, strong verification—plus one cautionary diff where the story did not match the change.

Regular retrospectives

Talk about what worked, what mis-fired, and what to standardize—same as any practice, just name Bob explicitly.

Pair programming with Bob

One person holds intent; both read plans and diffs. Knowledge spreads without turning Bob into a solo sport.

Make it optional but visible

Usage can stay voluntary; documentation of usage should not be. If Bob helped, say so in the trail—if it did not, that is fine too.

13.7. Sharing Custom Modes and Rules

Consistency comes from shared steering artifacts—modes, rules, a tight `AGENTS.md`—not from hoping everyone picks the same adjectives.

Mechanics and layering live in Skills, modes, and rules as team memory. From a team angle the rules are smaller:

- Memory people cannot find does not exist for onboarding.
- Memory reviewers do not trust gets bypassed.
- Version and review it like product code; retire it when the system or habit changes.

A short `AGENTS.md`, a handful of real rules, and one or two commands people actually run usually beat a fat catalog nobody opens.

13.8. Team Patterns for Bob Usage

Names for workflows differ; the underlying habits do not. Most healthy teams settle on three boring ones:

- **Analysis before non-trivial change** — use Bob to surface dependencies, existing behavior, and likely impact before anyone edits code.
- **Shared plan before execution** — for risky or wide changes, make the plan visible in a PR draft, ticket, or team chat before the diff lands.
- **Evidence in the review trail** — when Bob found a risk, helped preserve behavior, or changed the shape of the work, leave that evidence where reviewers can inspect it later.

Different teams will formalize those habits differently. The point is not to make every squad look identical. The point is that nobody should have to reverse-engineer the process from the final diff alone.

13.9. Example: A Team Using Bob Together

Picture a team modernizing a legacy authentication service with weak tests and high blast radius.

The team lead starts by asking Bob for analysis, not code, and shares the findings. One developer turns that into a preservation plan. Another uses Bob to identify missing tests. Reviewers check the final diff against the stated contract and the shared plan instead of reading it as unexplained magic. When staging reveals an OAuth edge case, the fix becomes another documented learning, not just a patch.

Nothing in that story depends on everyone using Bob the same way. It works because intent, plans, verification, and learning stay visible across the team.

13.10. Rollout and governance

Pilot teams, phased adoption, and lightweight KPIs beat “flip a switch for everyone Monday.” Start with verification-heavy workflows—plans in tickets, reviewers who understand what Bob was allowed to see—and expand access only after containment patterns hold (modes, MCP lists, `.bobignore`). Intent beats slogans: what are you proving in the pilot, who signs off when automation touches customer data, and how do you know the rollout still matches policy ninety days later?

If those KPIs turn into prompt counts, generated lines, or guessed hours saved, you are measuring theater. Measuring Bob Without Measuring Theater is the follow-on chapter for that problem.

For a ninety-day arc, governance checkpoints, and measurement ideas aimed at leads and enablement folks, pair this chapter’s collaboration tactics with IBM Bob documentation as your org rolls out access.

13.11. What You Learned

Teams win when intent, plans, and findings show up in commits and PRs—speed without visibility is a private win, not a team one. What Bob should not do lists the guardrails (deployments, blind approvals, context-free asks) that erode under schedule pressure.

Try one small change in the open: paste the plan on the first PR; the awkward part is the habit, not the typing.

Chapter 14. Roles in the Agentic SDLC

14.1. Same Loop, Different Leverage

Bob is easy to misunderstand as a developer-only tool because the visible surface is an IDE.

That is too small a frame.

The work around an agent is broader than code generation. Intent, constraints, evidence, review, and permissions all shape the outcome. Different roles influence different parts of that loop.

This chapter is not a set of tutorials.

It is a translation guide for mixed-role teams.

14.2. For Developers

Your job shifts away from typing every implementation detail by hand and toward steering safe change.

You still write code. You still review diffs. You still own behavior.

What changes is the center of gravity:

- frame the task clearly
- show Bob the right evidence
- review the plan before execution
- preserve existing behavior where required
- reject plausible but risky shortcuts

The developer role becomes more editorial and less purely mechanical.

14.3. For Architects

Your leverage increases when constraints become executable.

Architecture is no longer only a diagram or a decision record. It becomes part of the runtime discipline around the agent:

- rules
- modes
- skills
- approval boundaries
- accepted patterns
- explicit trade-offs

If a principle matters, the team should not rely on memory alone. Bob should be able to see it in the environment the same way a human reviewer can.

14.4. For Product Managers

Your job is not to become the team's prompt engineer.

Your job is to make intent testable.

That means being precise about:

- the user outcome
- the non-goals
- the business constraints
- the trade-offs the team is allowed to make
- the evidence that proves success

Vague requests become vague plans. In agentic work, ambiguity scales faster because assumptions can turn into edits.

14.5. For Testers and QA

Your leverage increases, not decreases.

Before Bob changes code, someone needs to define:

- the expected behavior
- the regression risks

- the edge cases
- the failure scenarios
- the stop conditions when behavior is no longer trustworthy

That is contract work. It is exactly what keeps a plausible implementation from becoming a production defect.

14.6. For Security and Compliance

You are no longer only reviewing code at the end.

You shape the environment in which the agent operates:

- permission levels
- MCP trust boundaries
- data-handling rules
- evidence retention
- approval requirements
- escalation paths

Least privilege, auditability, and classification rules are not side notes. They are part of the task design.

14.7. For Engineering Managers

The useful question is not "who on the team uses AI?"

The better questions are:

- which workflows are visible
- which habits are repeatable
- which review expectations are documented
- which permissions are justified
- which practices are safe enough to scale

Management in an agentic SDLC is partly about turning clever individual behavior into boring team behavior.

14.8. What This Chapter Is Not Saying

Not everyone needs to live in the same interface every day.

Not everyone needs to approve shell commands.

Not everyone needs to read diffs at the same level of depth.

But everyone involved in software delivery should understand where their role influences intent, constraints, verification, or evidence.

14.9. What You Learned

Bob does not collapse every role into "person who prompts." Developers, architects, PMs, QA, security, and managers all shape different parts of the same loop. The work becomes safer when those responsibilities are explicit instead of implied.

The Bob mindset closes the human side of the book by focusing on the habits that carry across tools, versions, and teams.

Chapter 15. Skills, Modes, and Rules as Team Memory

15.1. Two Levers You Did Not Have to Author

Modes and slash commands are the shortest path out of retyping the same setup paragraph.

- **Modes** cap what Bob may do for the rest of the chat—read-only help versus edits versus full builder posture.
- **Slash commands** drop in a ready-made prompt—your team can ship commands under `.bob/commands/` in the repo or under `~/.bob/commands/` for yourself.
- **Rules and project briefs** hold the slower-moving stuff: `AGENTS.md`, `.bob/rules/`, and other repo-local text that says what “normal” means here.

Most days you are not authoring those layers—you are discovering what already exists and pairing mode with command for the task. Bob in your IDE shows where that lands in the UI; this chapter is the habit layer that turns one-off prompts into something the team can reuse.

Official reference: Modes and Slash commands in IBM Bob documentation.

15.2. Finding Commands and Switching Mode

Type a slash (`/`) in the chat input. A unified menu lists mode switches at the top (Ask, Plan, Code, Advanced, Orchestrator, plus any custom modes your organization provides), then built-in commands such as `/review`, then team commands from the project, then your personal commands. Fuzzy search works as soon as you type another character after `/`—narrow the list by keyword and read the short description beside each entry before you send.

If a command shows an argument hint such as `<base-branch>`, type that value after the command, separated by a space—the hint documents shape; it does not fill itself in.

Pick mode before you polish the sentence so tool access matches the job: Ask for explanations without edits; Plan when you want a strategy before commits; Code when the repo and shell are enough; Advanced when you need browser, MCP, or skills. Orchestrator is for work that actually spans specialties—it coordinates subtasks instead of forcing one mode to pretend it owns everything.

On macOS use ⌘. ; on Windows/Linux use Ctrl. to cycle modes when you already know

the posture.

15.3. Skills: Packaged Workflows You Trigger With Words

Skills are workflows someone already packaged—reviews, migration checks, doc passes. Using them does not mean editing `SKILL.md`; it means phrasing the task so Bob can match the skill description.

- **Advanced mode.** Skills assume the broad tool surface—switch modes before you expect one to fire.
- **Approval.** Bob prompts before running a skill unless auto-approve says otherwise—confirm when scope matches, tighten settings when it does not.
- **One activation per chat.** Matching uses your words plus the skill text; if nothing fits, you get a generic answer. Name the deliverable (“security review of `src/api/auth.ts`”, “release notes for the last twenty commits”) instead of “help with this file.”

Resolution order matters: project `.bob/skills/…` beats the same name globally, then `~/.bob/skills/`, plus whatever extensions install. If you expected a skill and got boilerplate, ask whether the skill exists on disk and whether the ask was specific enough.

Official reference: Skills in IBM Bob documentation.

15.4. Skills Are Team Habits in Package Form

Treat skills as **team habits in package form**.

Worth packaging when the team keeps paying the same tax:

- review checklists that slip
- release-note steps that change per author
- modernization sequences everyone re-explains
- ADR drafts that always start from zero

When the pattern is real and repeating, a skill moves memory into something discoverable—infrastructure for habits, not automation theater.

15.5. The Skill Lifecycle

A healthy lifecycle looks like this:

1. Notice repeated work or repeated confusion.
2. Capture the current best workflow in plain language.
3. Turn it into a skill, command, mode, or rule only after the pattern is stable enough to be worth sharing.
4. Test it on real examples.
5. Document when to use it and when not to.
6. Review it like code.
7. Retire it when the team or product changes.

That last step matters. A stale skill is not neutral—it is an outdated habit with better packaging.

15.6. Choose the Right Memory Layer

Same workflow, wrong bucket, and the team either loses the habit or automates past the point where judgment still belongs.

- **Slash commands** — repeat asks you want one keystroke away.
- **Modes** — posture and permissions for whole threads.
- **Rules** — constraints that should hold every turn.
- **Skills** — multi-step flows with tools and review expectations baked in.
- **`AGENTS.md`** — the short map that points people at the rest.

Wrong layer usually shows up as silence (“nobody remembers”) or false confidence (“the skill did it”).

15.7. Bob Can Draft the Decision Packet, Not Make the Decision

Architecture is the obvious place where “generate text” can masquerade as “decided.”

Bob should not own the call—but it **can** assemble the packet humans actually review:

- problem
- context
- constraints
- options
- trade-offs
- risks
- recommended path
- evidence used
- decision owner
- review date

That shortens the blank page without moving accountability off the people who sign.

15.8. A Straightforward Loop

1. Pick the mode that fits the work.
2. Type `/`, search the command, add arguments if the hint calls for them, send.
3. Stay in the thread for follow-ups—the command seeds context for that conversation; mode still caps tools.

Mode is permission; the command is the ask for that message. The same `/pr-prep` can summarize in Ask or touch files in Code—that split is deliberate.

15.9. What You Learned

Mode first, then words: posture sets permissions, slash commands replay what the team already wrote down, skills fire when language matches the package. The artifacts are where judgment gets shared—not where it disappears.

Go deeper in IBM Bob documentation:

- Slash commands

- Modes
- Skills

Modernization without behavior drift shifts to code you did not write—same careful reading, harder preservation story.

Chapter 16. Measuring Bob Without Measuring Theater

16.1. Teams Will Ask for Numbers

Sooner or later, every rollout reaches the same question:

"How do we know this is working?"

That is a fair question.

It becomes a bad question when the only acceptable answer is a number that flatters the tool.

Good measurement helps teams learn. Bad measurement turns into theater around prompts, token counts, and guessed time savings no one can audit.

16.2. What Not to Measure

These numbers are easy to collect and easy to misread:

- number of prompts
- number of generated lines
- raw token consumption
- hours "saved" based on guesses
- number of people who tried the tool once

None of these tells you whether the work became safer, clearer, or easier to review.

They mostly tell you that activity happened.

16.3. Measure the Delivery System, Not the Demo

The useful unit is not "how much Bob talked."

It is "what changed in delivery quality, reviewability, and learning."

Better measures look like this:

- review cycle time for non-trivial changes

- defects found before merge
- regression rate after Bob-assisted changes
- time to understand unfamiliar code well enough to act
- percentage of PRs with clear intent and verification evidence
- repeated onboarding questions that disappear after workflows are documented
- reusable team workflows created, used, and retired

Those measures are less glamorous and more honest.

16.4. Measure by Workflow

One rollout rarely improves everything at once.

Measure by workflow instead:

- code review
- legacy modernization
- onboarding and codebase explanation
- documentation support
- security review preparation

That keeps the comparison fair. A team might see immediate gains in review quality and almost none yet in architecture work. That is normal.

16.5. A Small Starter Scorecard

If the team is early, start with a short scorecard:

- Did the PR state intent clearly?
- Did the PR include verification evidence?
- Did Bob catch a real issue before merge?
- Did the reviewer understand what was changed without reverse-engineering the whole chat?

- Did the workflow save understanding time on unfamiliar code?

These are not vanity metrics. They are leading indicators that the process is becoming legible.

16.6. Baseline Before You Celebrate

The easiest way to fool yourself is to start measuring only after the rollout begins.

Take a baseline first where you can:

- how long non-trivial reviews usually take
- how often regressions slip through
- how long onboarding questions keep repeating
- how often PRs arrive without clear intent or verification

Then compare against that.

Without a baseline, every success story sounds convincing and none of them are anchored.

16.7. Beware Goodhart in Agentic Form

The moment a metric becomes a target, people will optimize for the metric instead of the outcome.

Examples:

- "Every PR must mention Bob" becomes ritual text no reviewer trusts.
- "Use Bob on every ticket" pushes the tool into trivial work it does not improve.
- "Lower review cycle time" pressures reviewers to approve faster instead of reviewing better.

Measurement should expose learning, not force cosmetic compliance.

16.8. What Mature Teams Learn to Measure

As the rollout matures, the interesting signals shift.

Later-stage questions include:

- Which skills and rules are actually reused?
- Which modes or workflows create fewer surprises?
- Which kinds of escalated permissions remain rare and justified?
- Which practices had to be retired because they caused confusion or drift?
- Which patterns reduced repeated explanations across teams?

That is when measurement stops being a dashboard for the tool and becomes feedback for the organization.

16.9. The Point Is Learning, Not Proof by Spreadsheet

You do not need an elaborate program on day one.

You need enough evidence to answer:

- Is the work more reviewable?
- Are we catching important issues earlier?
- Are people understanding unfamiliar systems faster?
- Are the shared habits getting more boring and repeatable?

If the answer is no, more prompts will not save you.

16.10. What You Learned

Measuring Bob well means measuring the delivery system, not the novelty of the tool. Ignore activity theater. Track whether intent, verification, reviewability, risk reduction, and team learning are improving in workflows that actually matter.

Keeping Bob boring closes the book with the habits that make those measurements worth trusting.

Chapter 17. The Bob Mindset

17.1. Getting Value Beyond the Tool

You already know how to drive Bob—panels, phases, `/review`, boundaries.

The durable payoff is not keystrokes; it is how you frame work before the model prints anything.

This chapter is habits: problem framing, systems view, questions that expose limits, and responsibility that does not slide because the answer sounded polished.

The assistant changes; the posture stays with you.

17.2. The Shift from Code to Systems

Without deliberate framing, work collapses to snippets. With it, you chase intent and blast radius first.

The old way

Task arrives → think in classes and lines → implement → test → commit. Fast to start; easy to ship the wrong abstraction.

The new way

Same task → map what exists, what must stay true, what could break → intent, evidence, decision → then edits.

Typing lands later; thinking lands earlier. That costs clock time up front and buys safer moves later.

Why this matters

Files are cheap to edit; systems are expensive to misunderstand.

Optimize for typing speed and you get clever patches. Optimize for change safety and you ask different questions—often the ones Bob surfaces when you do not skip planning.

Example: The system thinker

Narrow framing:

> *"I need to add a method to UserService. I need to call the database. I need to return the result."*

Wider framing:

> *"I need to add a feature. What does this feature affect? What services does it touch? What data does it need? What could break? What tests are needed?"*

Same eventual code path—different amount of surprise when production disagrees with your assumptions.

17.3. Use the Earlier Chapters on Purpose

By this point, intent, context, contracts, permissions, and verification should not feel like separate topics.

They are what the mindset looks like in practice.

If you skip them and ask Bob to improvise from a vague sentence, the tool will reflect that gap right back at you. If you use them deliberately, the interaction gets calmer, the plans get sharper, and the diffs become easier to defend.

17.4. Better Questions, Better Boundaries

The fastest way to improve Bob’s answers is not to sound more "AI native." It is to ask questions that expose stakes and limits.

Questions worth asking usually do one of three things:

- reveal impact: "What would break if we change this public method?"
- reveal trade-offs: "What are the safer options if we cannot change the schema?"
- reveal risk: "Which assumptions in this plan are not backed by tests yet?"

Those questions keep the work in the space where judgment still lives.

17.5. Systems Before Snippets

The mindset shift is simple to say and easy to forget:

Do not start by asking Bob for isolated code when the real problem lives in a system.

Start by asking:

- what already exists
- what pattern the team already follows
- what behavior must be preserved
- what could break around the edge of the change

That is slower than grabbing a snippet. It is also how you stop a plausible answer from becoming maintenance debt.

17.6. Using AI Without Giving Up Responsibility

Models err sometimes; bigger failure mode is you switching off because the tone sounded confident.

The responsibility trap

Smooth prose plus green tests invites autopilot approval—exactly when someone still has to own paging policy and customer impact.

How to stay responsible

Understand the suggestion — If you cannot explain it, you are not done reading.

Evaluate trade-offs — Default answers hide alternatives; ask for them.

Own the outcome — Your merge; your incident if you guessed wrong.

The responsibility checklist

Before approving:

- Do I understand what this does?
- Do I know why this approach was chosen?
- Do I know what could go wrong?
- Can I explain this to my team?

- Am I comfortable with this in production?

Any "no" means more questions or a smaller step—not a rubber stamp.

Example: Staying responsible

Caching looks obvious until invalidation, memory caps, and consistency show up. Walk the checklist; if comfort only arrives after Bob spells trade-offs and alternatives, that delay is the point—not friction.

Ask directly: trade-offs, invalidation paths, footprint, other designs. Then decide with eyes open.

17.7. What the Mindset Changes Day to Day

Behavior shifts more than philosophy:

- Lead with analysis more often than with edits.
- Pull plans before execution when stakes warrant it.
- State constraints early so suggestions stop fantasizing.
- Send back plausible diffs that skip reasoning.
- Write enough trail that the next reader is not guessing.

That is habit, not a poster on the wall.

17.8. The Mindset Is Transferable

Same posture applies to other assistants, human reviewers, and plain debugging: clarity of goal, evidence on the table, explicit trade-offs, explicit ownership.

17.9. What You Learned

Systems before snippets; intent you could test; questions that surface limits; responsibility that does not evaporate because the summary sounded confident. Bob is not magic—it amplifies crisp asks and exposes fuzzy ones.

Research on developers learning with AI backs the same idea: the variable is cognitive effort—conceptual questions, hybrid code-plus-explanation, or generate-then-comprehend patterns preserve understanding better than passive copy-paste.

Keeping Bob boring turns this into weekly habits—start small, document patterns, let the workflow go boring.

Interfaces churn; judgment does not. Keep your bar for approval higher than the automation dial's default.

Chapter 18. Keeping Bob Boring

18.1. Where to Go from Here

End of the book is a bad place to invent new tricks.

The useful move is to keep the workflow dull enough that you stop gambling on novelty—same loops, same checks, same trust bar.

From here: a small weekly ramp, a few workflows worth repeating, and what I mean by advanced once the small loops actually stick.

18.2. Start Small, Build Habits

Layer one thing at a time. Piling every panel and slash command into week one mostly trains panic, not judgment.

Week 1: Analysis only

Use Bob for read-only work: explain behavior, trace failure, map blast radius. No edits yet.

- "Analyze this service and explain what it does"
- "Explain why this test is failing"
- "What would break if I change this method?"

Goal: evidence before keystrokes, and systems-shaped questions before patches.

Week 2: Add code review

Before you commit, run review and read what comes back.

- Run `/review` before every commit
- Read Bob’s findings
- Fix issues Bob identifies
- Document what Bob found

Goal: catch dumb mistakes while the diff is still cheap; leave a short trail of what you fixed and why.

Week 3: Add planning

For non-trivial changes, nail intent first, then let Bob propose a plan.

- Define clear intent
- Ask Bob to create a plan
- Review the plan
- Approve and execute

Goal: deliberate steps you could defend in review—not a surprise refactor.

Week 4: Full workflow

Run analysis → plan → execute → review as one rhythm.

If week four still feels foreign, stay on week three longer. The calendar is a suggestion; the habit is the point.

Why start small

Small slices beat a heroic rewrite of how you work. Confidence comes from repetition, not from one intense Monday.

18.3. Common Workflows to Explore Next

At this point, do not try to adopt ten workflows at once.

Pick one and make it boring.

Good candidates are:

- legacy-code exploration when the codebase is unfamiliar
- refactoring with preservation intent when behavior matters
- security-first review before merge
- onboarding and explanation work for new teammates

Each of those already has a home in the earlier chapters. The win now is repetition, not collecting more patterns than the team can remember.

18.4. Advanced Use Cases

Save the wider combinations for later.

Multi-service coordination, architecture support, compliance-heavy reporting, and cross-team rollout work all benefit from Bob, but only after the basic loops hold: bounded context, explicit contracts, visible plans, and trustworthy review evidence.

The trap is reaching for advanced use cases because they sound strategic. The healthier path is to earn them by making the smaller loops reliable first.

18.5. When Bob Shines the Most

Bob pays best when the job is map-then-change, not hero typing.

Scenario 1: Changing unfamiliar code

You need to edit what you do not fully own yet. Use Bob to summarize behavior, dependencies, and risk before you touch lines. That is the highest-leverage read path.

Scenario 2: Coordinating complex changes

Many files or services move together. Bob helps trace impacts and keep the plan from silently dropping an edge case.

Scenario 3: Maintaining consistency

Team conventions already exist; Bob is decent at matching style and naming when you point it at real examples.

Scenario 4: Catching issues early

Review and security-oriented passes belong here—find the sharp edges before merge, not from a pager.

Scenario 5: Learning and knowledge transfer

Explanations, examples, and pointers to docs speed onboarding when someone still has to run the code themselves.

When Bob doesn’t shine

Reach for the keyboard alone when:

- You’re learning something new (write code yourself)
- You’re prototyping quickly (speed matters more than safety)
- You’re in an emergency (no time for deliberation)
- You’re making architectural decisions (requires deep judgment)
- You’re working on trivial changes (overhead not worth it)

Pick the mode on purpose, not because the chat panel is already open.

18.6. How to Keep Bob Boring, and Your Systems Reliable

Excitement in an assistant usually means variance. For production work I want the opposite: same sequence, same gates, same surprise surface.

Boring is good because you can predict what you asked for, spot when the answer drifted, and stop relying on hero fixes after the fact.

How to keep Bob boring

Use consistent workflows

Pick a few paths (analyze → plan → implement → review) and run them until muscle memory shows up. Change them slowly; whims are how teams lose shared expectations.

Document your patterns

Modes, rules, and team conventions belong in writing so onboarding is not folklore and your future self does not improvise from scratch every month.

Review and improve regularly

Once a month is enough: what worked, what felt flaky, what one tweak would remove the most thrash. Small edits beat dramatic reinventions.

Share with your team

When something works—or fails—say so. Shared habits beat isolated brilliance.

Resist the temptation to be clever

Creative prompting is fun for demos; boring prompting ships. Straight paths beat stunt

workflows when someone else has to maintain the outcome.

The boring tool paradox

The more predictable Bob feels in your hands, the more you can lean on it for real constraints instead of treating every session like a product launch.

Boring here means you are doing software work with the tool, not performing with it.

18.7. Your Next Steps

Concrete cadence beats ambition lists:

This week

1. Pick one workflow from this chapter
2. Try it on a real task
3. Document what worked and what didn’t
4. Refine the workflow

This month

1. Build habits with the basic workflows
2. Create custom rules for your team
3. Share your learnings with colleagues
4. Review and improve your usage

This quarter

1. Master the basic workflows
2. Explore one advanced use case
3. Develop team patterns
4. Measure the impact using delivery evidence, not prompt theater

This year

1. Make Bob usage boring and predictable
2. Build a library of team workflows
3. Onboard new team members with Bob
4. Continuously improve your approach

The long term

Expertise in Bob is not the prize; better teammate judgment is.

Assistants ship and sunset; habits stick:

- Thinking in systems
- Defining clear intent
- Examining evidence
- Making informed decisions
- Staying responsible

Bob was the gym where you rehearsed those; it does not own them.

18.8. Final Thoughts

Same premise as early chapters: code is cheap, software is not. Use Bob to tighten decisions under real constraints—not to skip deliberation or outsource judgment when production calls.

Keep the tool boring. Keep the skills yours. Build something that lasts.

If you want Bob in your own environment, IBM runs a time-limited trial: you get 40 Bobcoins for 30 days, with Bob IDE and Bob Shell available from the same place. When the trial ends, you can move to a paid plan (Pro, Pro+, or Ultra) if you are continuing. Details and signup: Start a free trial.

Appendices

Appendix A: About the Author

Markus Eisele helps Java teams turn AI hype into production-ready systems.

Markus has more than 25 years of experience working with large enterprises. He has helped organizations modernize Java platforms. From heavy, hard-to-change monoliths to cloud-native, container-first architectures built with Quarkus and Kubernetes. He focuses on real systems. Systems that are fast to change. Safe to operate. And boring in production.

Today, Markus works at IBM Research as a Principal Product Manager. He bridges research and product work with practical solutions for developers and platform teams worldwide.

He is a Java Champion, long-time community leader, and a trusted voice in the enterprise Java ecosystem.

Markus is known for making complex topics practical. He specializes in AI-infused Java architectures, local and enterprise-ready AI models, developer productivity, and platform engineering. His work shows how open source, Java, and AI come together in ways that scale across teams, organizations, and industries.

He is the author of Modernizing Enterprise Java (O'Reilly) and Applied AI for Enterprise Java Development (O'Reilly). He's a frequent international speaker. And he publishes The Main Thread, a Substack read by senior developers and architects for deep, hands-on guidance on Quarkus, enterprise AI, and modern Java architecture.

Markus brings credibility with developers and confidence to decision-makers. He helps organizations move faster while staying in control.

More: https://www.linkedin.com/in/markuseisele/ https://x.com/myfear

Appendix B: Changes by Version

Version	Summary	Date
1.2.0	Restructured the book around the agentic workflow.	May 2026
1.1.2	Minor changes to What Bob should not do	May 2026
1.1.1	Adding poster content.	April 2026
1.1.0	Extended to match posters.	March 2026
1.0.0	Original version.	December 2025